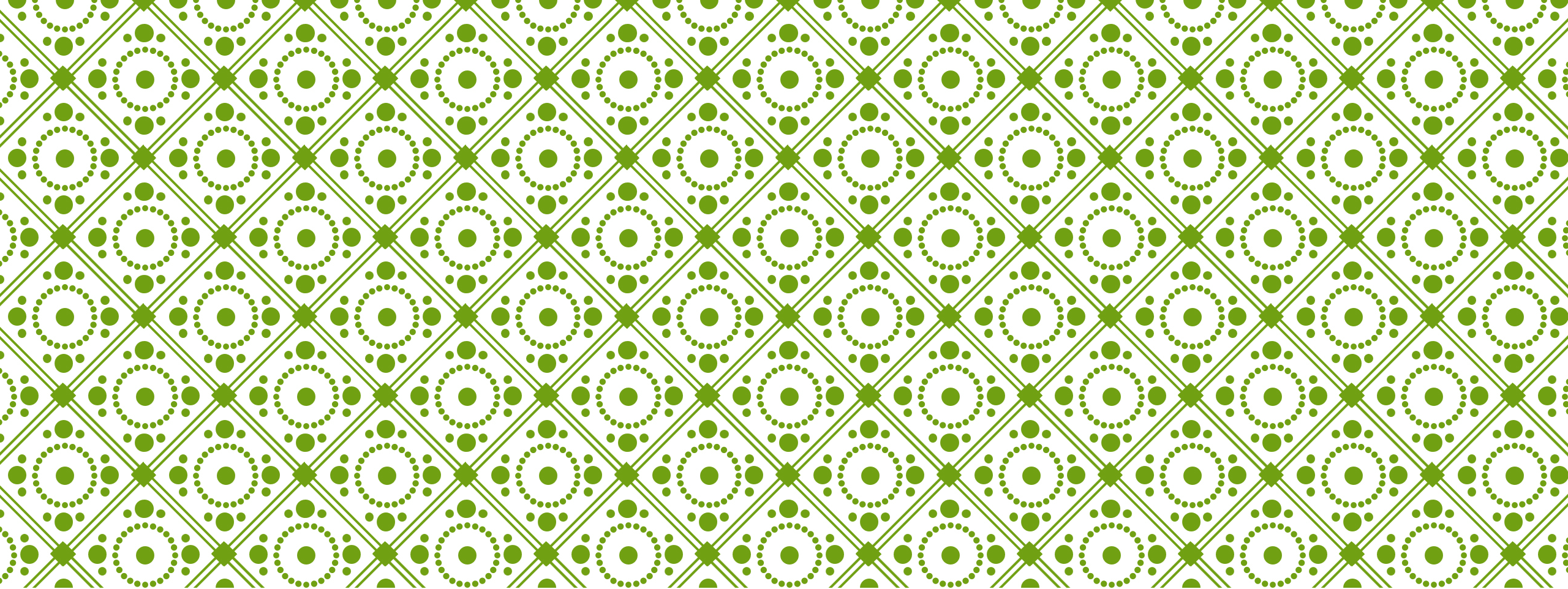




AI2002 – ARTIFICIAL INTELLIGENCE

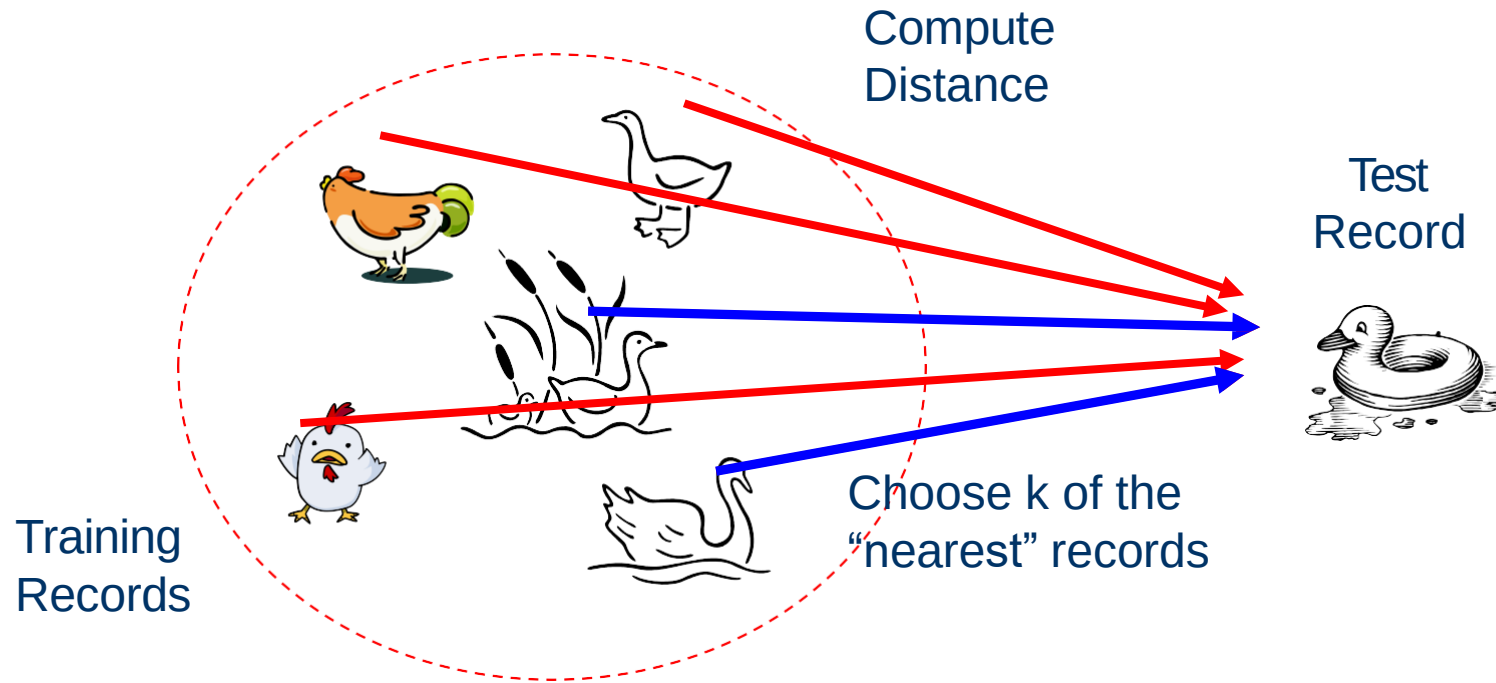
SPRING 2024 - LECTURE 40-42

Presented By: Mr. Sandesh
Kumar

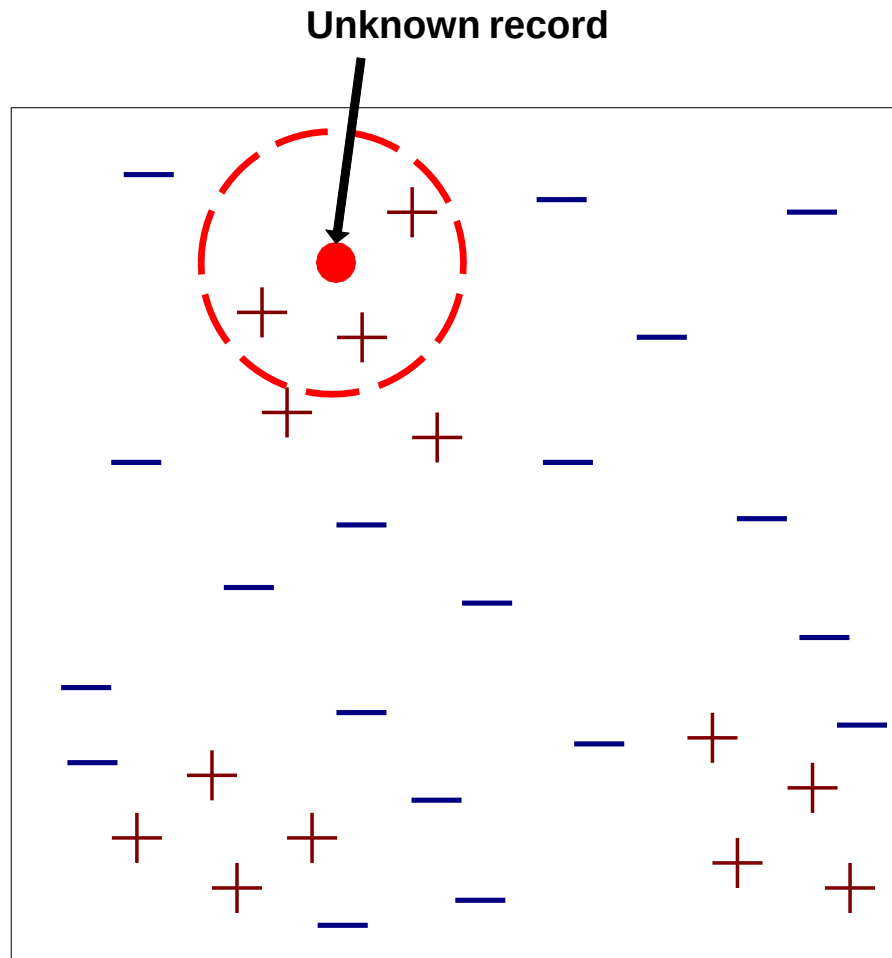
Slides are taken from University
of Pennsylvania

K-Nearest Neighbor Classifiers

- Basic idea:
 - If it walks like a duck, quacks like a duck, then it's probably a duck

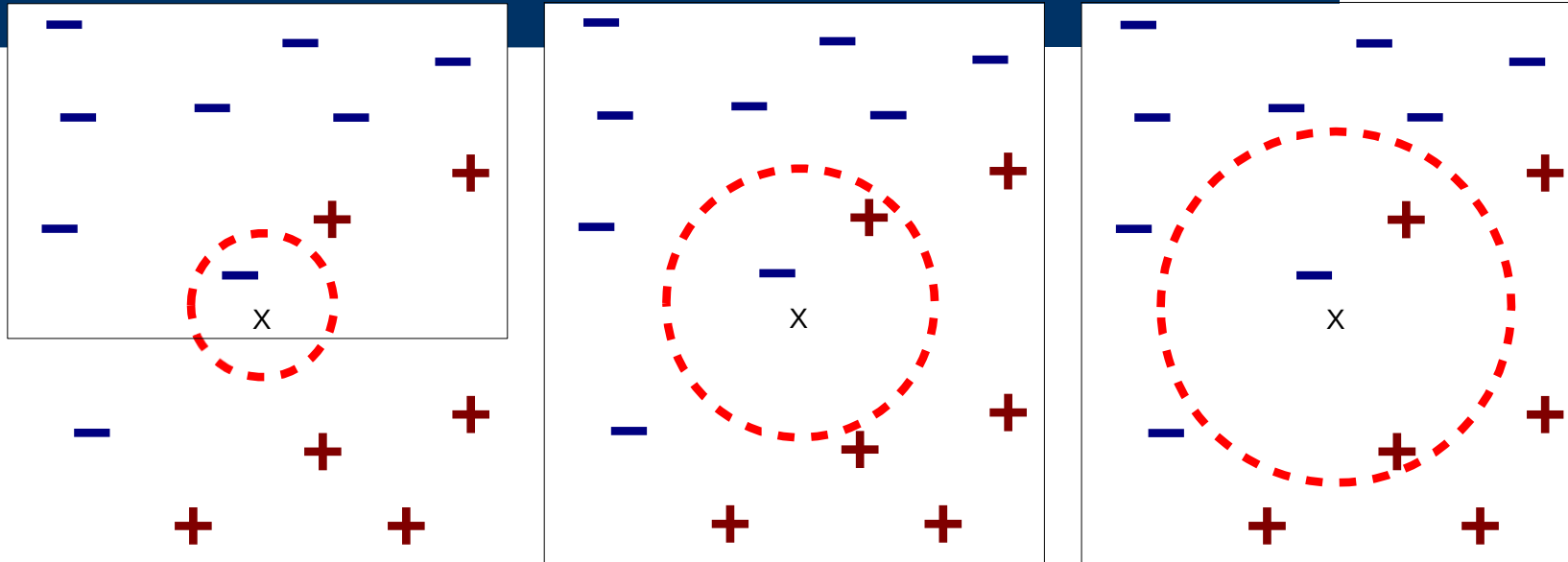


K-Nearest Neighbor Classifiers



- Requires three things
 - The set of stored records
 - Distance Metric to compute distance between records
 - The value of k , the number of nearest neighbors to retrieve
- To classify an unknown record:
 - Compute distance to other training records
 - Identify k nearest neighbors
 - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

Definition of K-Nearest Neighbor



(a) 1-nearest neighbor

(b) 2-nearest neighbor

(c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

K-Nearest Neighbor Classification

- Compute distance between two points:
 - Euclidean distance

$$d(p, q) = \sqrt{\sum_i (p_i - q_i)^2}$$

$$d(p, q) = \sum_i \text{abs}(p_i - q_i)$$

- Determine the class from nearest neighbor list
 - take the majority vote of class labels among the k-nearest neighbors
 - Weigh the vote according to distance
 - weight factor, $w = 1/d^2$

Example (KNN Classifier)

F1	F2	Class
1	5	0
0	8	0
0	6	1
1	2	1

Training Data

<i>1</i>	<i>3</i>	<i>?</i>
<i>1</i>	<i>4</i>	<i>?</i>
<i>0</i>	<i>3</i>	<i>?</i>
<i>0</i>	<i>4</i>	<i>?</i>

Test Data

Example (KNN Classifier)

Step 1: Computer Distance from Test Sample 1 to Training Data

Step 2:

Distance from Test Sample 1 to All Training Samples		Class
1	$ 1-1 + 3-5 = 0 + 2 = 2$	0
2	$ 1-0 + 3-8 = 1 + 5 = 6$	0
3	$ 1-0 + 3-6 = 1 + 3 = 4$	1
4	$ 1-1 + 3-2 = 0 + 1 = 1$	1

Step 3: Assign the Test Sample to Class with minimum Distance, Here is Class 1. So Test Sample 1 belongs to Class 1

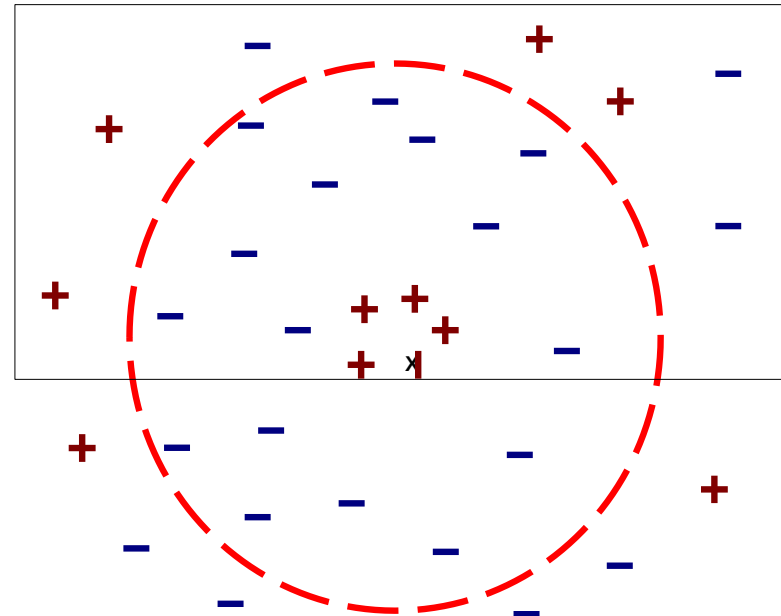
Example (KNN Classifier)

Exercise: Calculate for other 3 Test Samples

ID	Actual	Predicted
1	0	1
2	0	0
3	1	1
4	1	0 or 1

Nearest Neighbor Classification...

- Choosing the value of k :
 - If k is too small, sensitive to noise points
 - If k is too large, neighborhood may include points from other classes



Nearest Neighbor Classification...

- Scaling issues
 - Attributes may have to be scaled to prevent distance measures from being dominated by one of the attributes
 - Example:
 - height of a person may vary from 1.5m to 1.8m
 - weight of a person may vary from 90lb to 300lb
 - income of a person may vary from \$10K to \$1M

Normalize data

1. Find the Minimum and Maximum Values:

- Calculate the minimum value of X (let's call it $X_{\min}$).
- Calculate the maximum value of X (let's call it $X_{\max}$).

2. Apply the Min-Max Scaling Formula:

- For each data point X_i in your dataset, apply the following formula to normalize it to a value between 0 and 1:

$$X_{\text{normalized}} = \frac{X_i - X_{\min}}{X_{\max} - X_{\min}}$$

Here, $X_{\text{normalized}}$ is the normalized value of X_i .

By following these steps, your data will be scaled between 0 and 1. It's important to note that you should use the same $X_{\min}$ and $X_{\max}$ values for normalizing any new data points you want to predict or classify using the same model.

Example (NN Classifier)

Normalize Data from 0 to 1

F1	F2	Class
1	0.5	0
0	1	0
0	0.667	1
1	0	1

Training Data

<i>1</i>	<i>0.167</i>	<i>?</i>
<i>1</i>	<i>0.334</i>	<i>?</i>
<i>0</i>	<i>0.167</i>	<i>?</i>
<i>0</i>	<i>0.334</i>	<i>?</i>

Test Data

Example (NN Classifier)

After Normalization

ID	Actual	Predicted
1	0	1
2	0	0
3	1	1
4	1	1

REGRESSION

- Regression modeling is a process of determining a relationship between one or more independent variables and one dependent or output variable.
- Technique used for the modeling and analysis of numerical data
- Examples:
 - Predicting the height of a person given the age of the person.
 - Predicting the price of the car given the car model, year of manufacturing, mileage, engine capacity etc.
- Regression can be used for prediction, estimation, hypothesis testing, and modeling causal relationships

REGRESSION

- Based on the types of functions used to represent the relationship between the dependent or output variable and independent variables, the regression models are categorized into four types. They are:
 - Simple linear regression
 - Multiple regression
 - Polynomial regression
 - Logistic regression

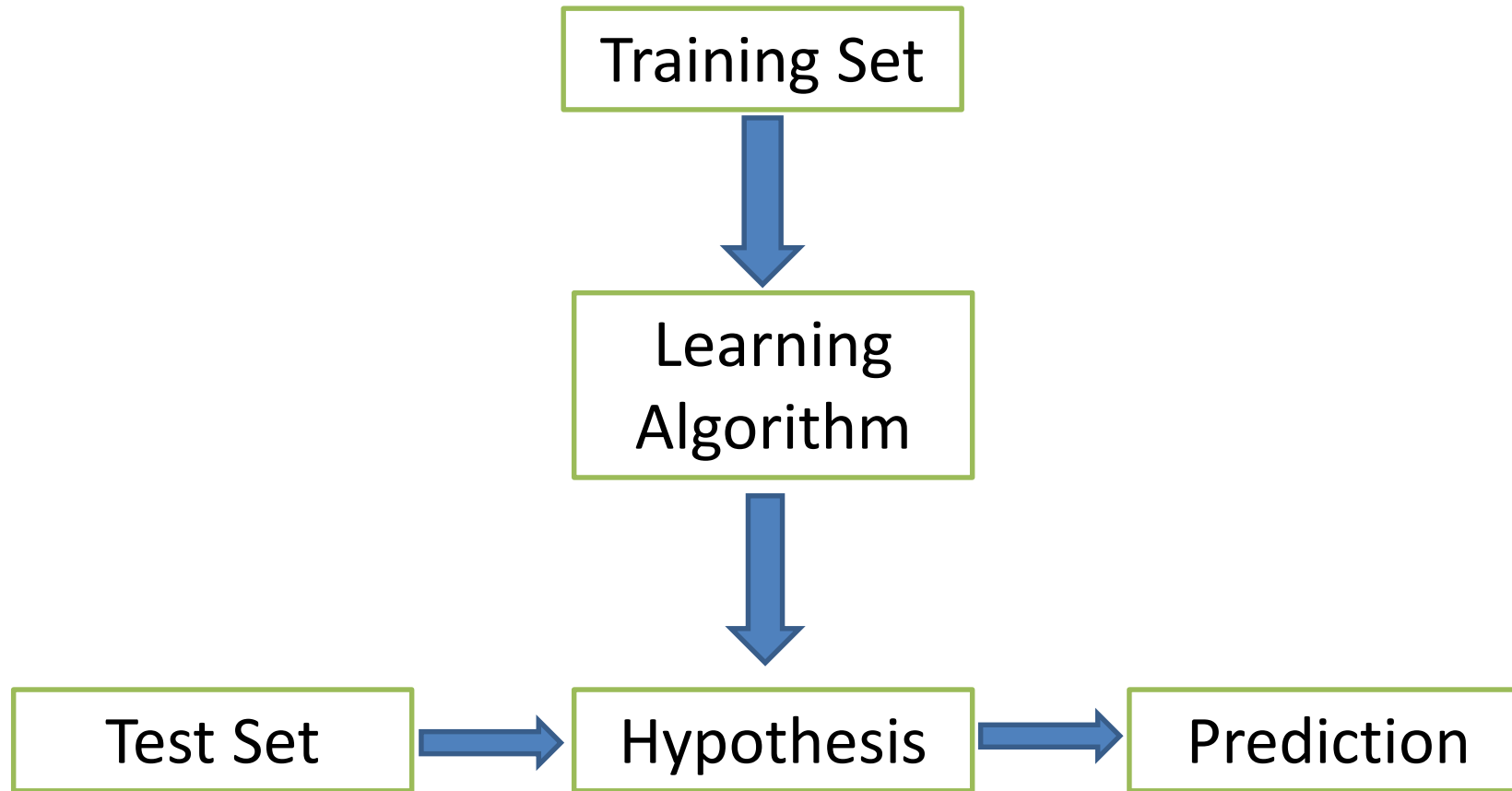
Regression versus Classification

- Classification: the output variable takes **class labels**
- Regression: the output variable takes **continuous values**

Examples

- Predicting House Value
 - Actual Price: £100,000
 - Predicted 1: £99,950 (Very Good Prediction)
 - Predicted 2: £50,000 (Very Bad Prediction)
- Predicting Car Premium
 - Using Location, Age, History etc

Recall the Learning Pipeline



Regression Lingo

$$Y = X_1 + X_2 + X_3$$


Dependent Variable

Independent Variable

Outcome Variable

Predictor Variable

Response Variable

Explanatory Variable

Linear Regression

Linear Regression

- **Linear Equation:** An **equation** that makes a straight line when it is graphed. Often written in the form $y = mx + b$. **Equation** of a Straight Line.
- **Linear regression** attempts to model the relationship between two variables by fitting a linear equation to observed data.

Wrong Interpretation!

- Wrong!!

A linear regression model assumes that the relationship between the **dependent variable** y_i and the **independent variables** x_i is linear.

- Correct!!

The linearity is **NOT** in terms of **x** but the **parameters** (coefficients).

Why Linear Regression?

- Suppose we want to model the dependent variable Y in terms of three predictors,

$$Y = f(X_1, X_2, X_3)$$

- Typically will not have enough data to try and directly estimate f
- Therefore, we usually have to assume that it has some restricted form, such as linear

$$Y = X_1 + X_2 + X_3$$

Logistic Regression

Logistic Regression

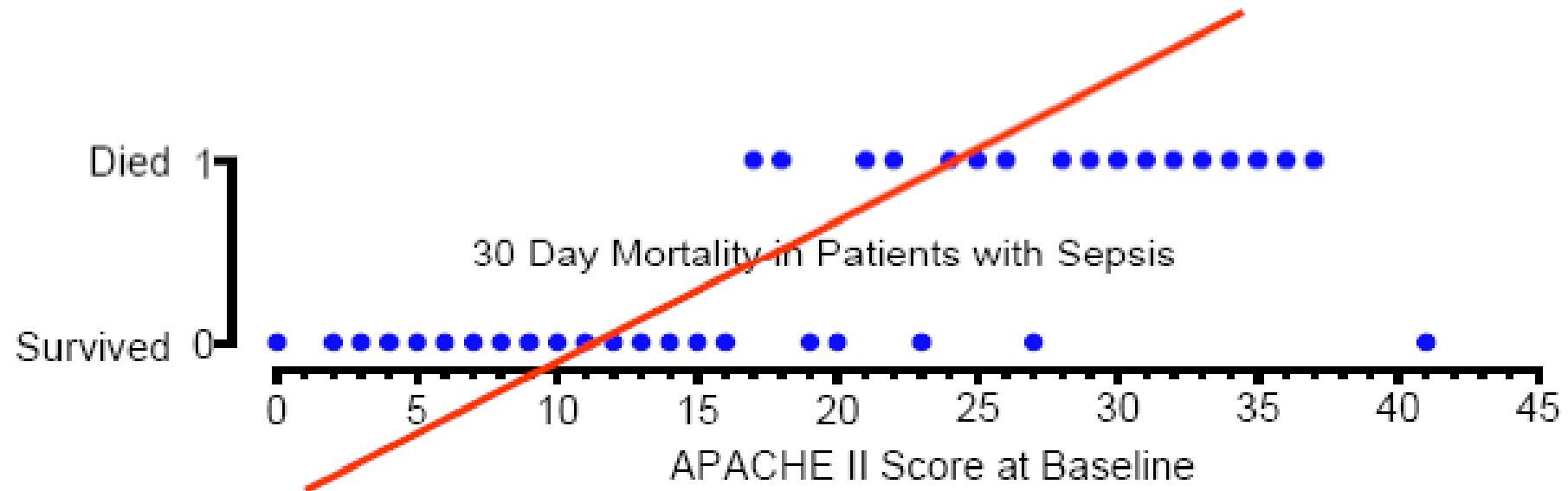
- Recall that linear regression assumes the response is quantitative
- In many cases, the response is qualitative (categorical).

Here, we study approaches for predicting qualitative responses, a process that is known as classification.

Predicting a qualitative response for an observation can be referred to as classifying that observation, since it involves assigning the observation to a category, or class.

Why not Linear Regression in Classification Problem

- Linear regression is unbounded



Logistic Regression Model

Want $0 \leq h_{\theta}(x) \leq 1$

- The hypothesis of linear regression $h_{\theta}(x)$ could produce probabilities less than zero or greater than one.
- To fix this, let's change the form for our hypotheses $h_{\theta}(x)$. We will choose

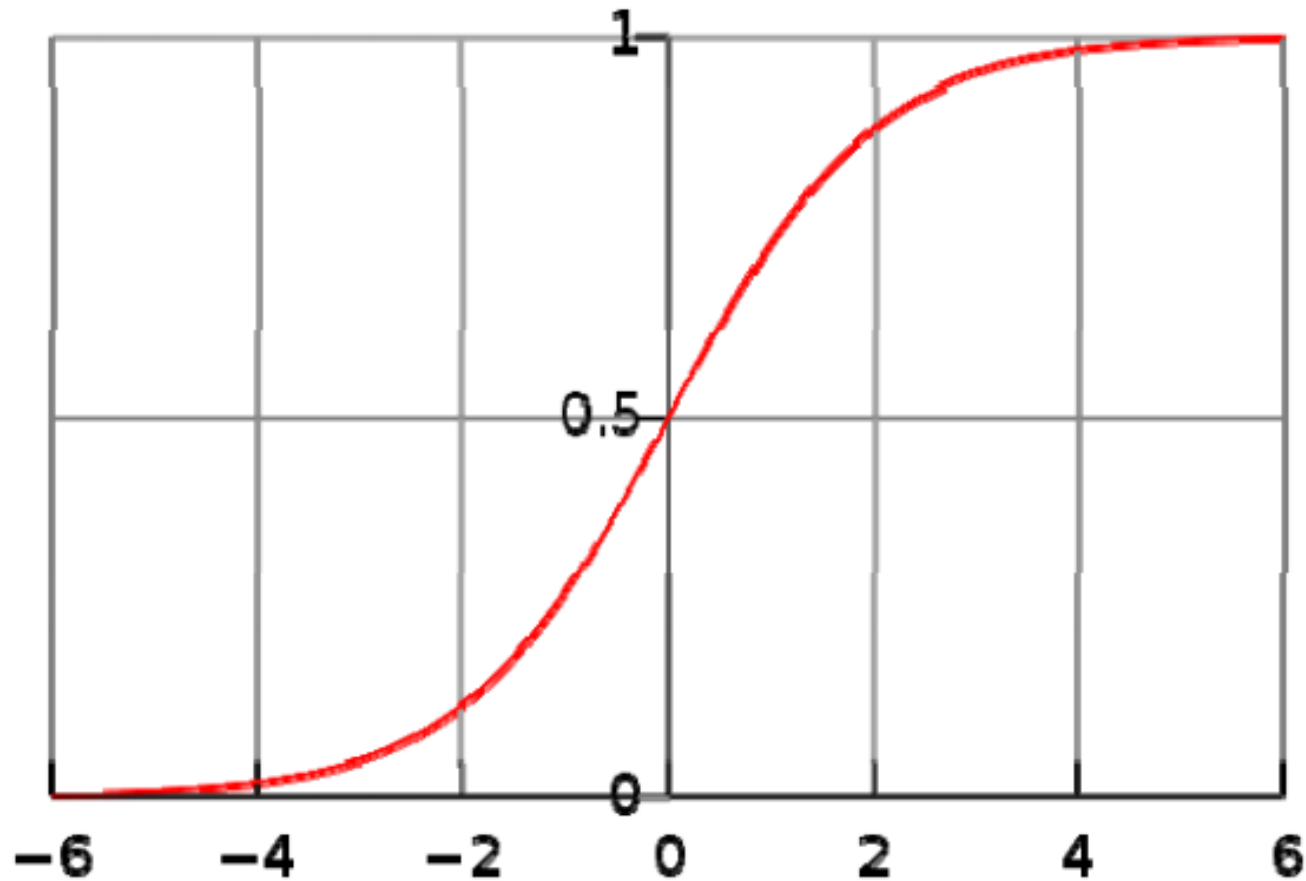
$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}},$$

where

$$g(z) = \frac{1}{1 + e^{-z}}$$

- is called the **logistic function** or the **sigmoid function**.

Standard logistic function



Using a logistic regression model

- Model consists of a vector β in d -dimensional feature space
- For a point $\mathbf{x}$ in feature space, project it onto β to convert it into a real number z in the range $-\infty$ to $+\infty$

$$z = \alpha + \beta \cdot \mathbf{x} = \alpha + \beta_1 x_1 + \cdots + \beta_d x_d$$

- Map z to the range 0 to 1 using the logistic function

$$p = 1 / (1 + e^{-z})$$

- Overall, logistic regression maps a point $\mathbf{x}$ in d -dimensional feature space to a value in the range 0 to 1

Using a logistic regression model

- Can interpret prediction from a logistic regression model as:
 - A probability of class membership
 - A class assignment, by applying threshold to probability
 - ◆ threshold represents decision boundary in feature space

Training a logistic regression model

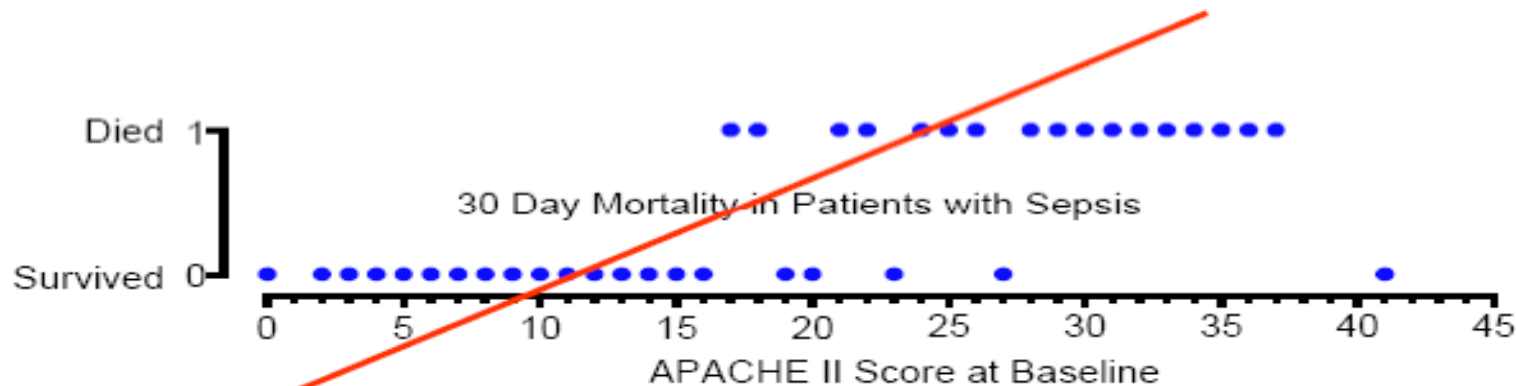
- Need to optimize β so the model gives the best possible reproduction of training set labels
 - Usually done by numerical approximation of maximum likelihood
 - On really large datasets, may use stochastic gradient descent

Logistic regression in one dimension

We wish to predict death from baseline APACHE II score in these patients.

Let $\pi(x)$ be the probability that a patient with score x will die.

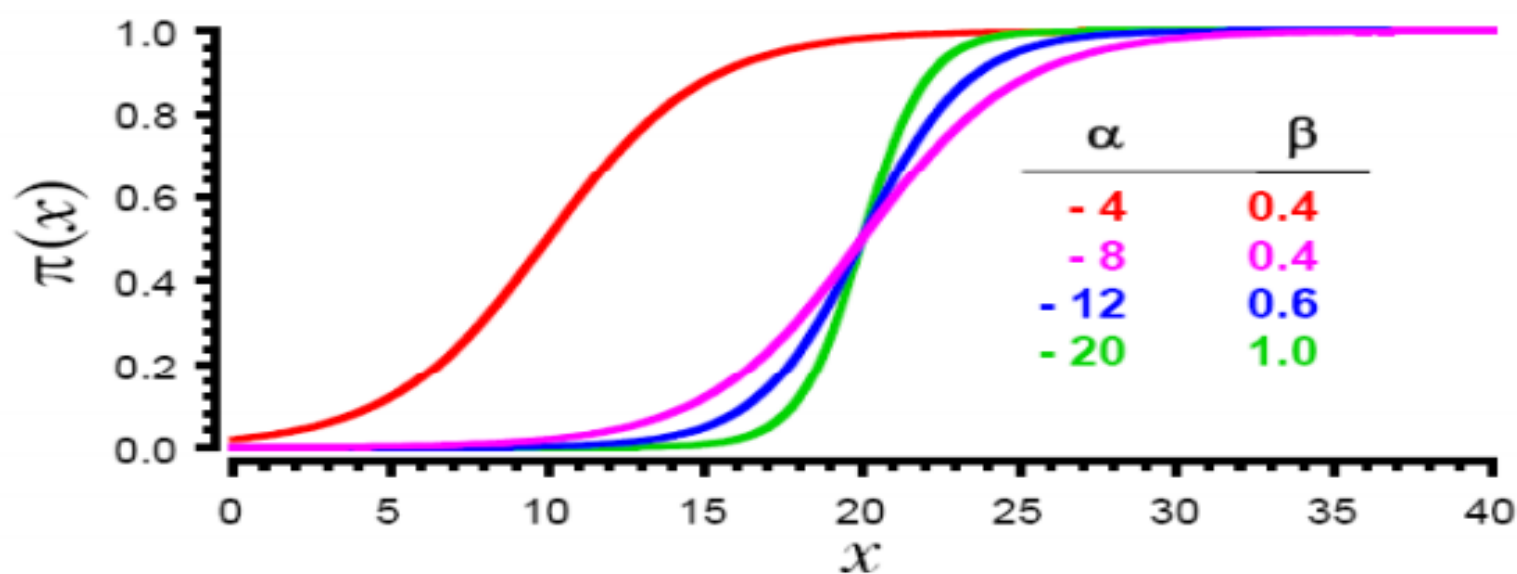
Note that linear regression would not work well here since it could produce probabilities less than zero or greater than one.



Logistic regression in one dimension

- Parameters control shape and location of sigmoid curve
 - α controls location of midpoint
 - β controls slope of rise

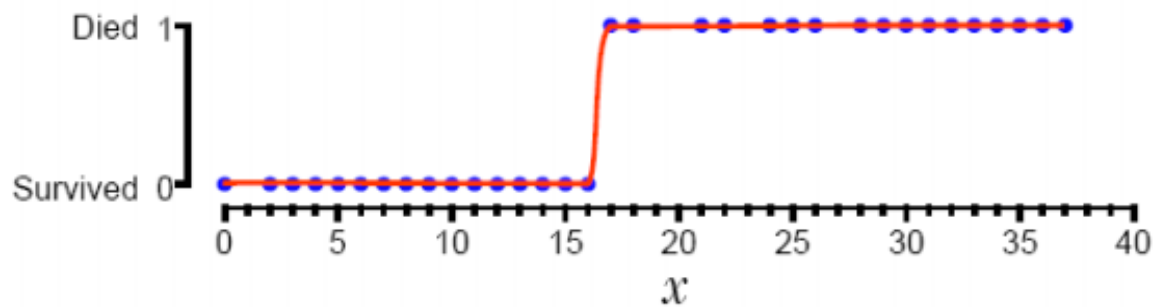
$$\pi(x) = \exp(\alpha + \beta x) / (1 + \exp(\alpha + \beta x))$$



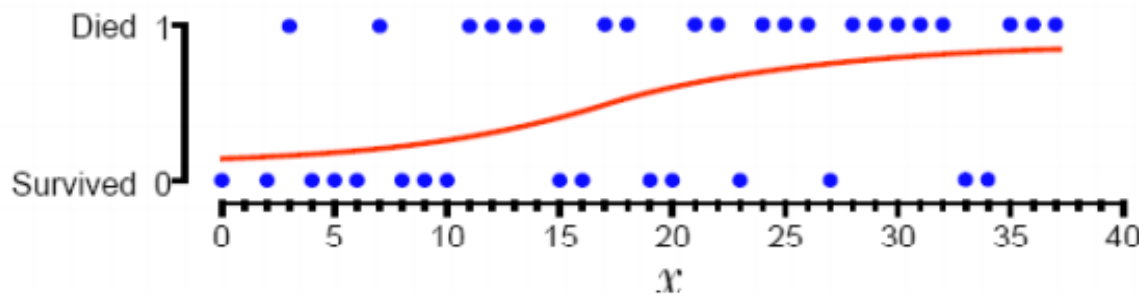
When $x = -\alpha / \beta$, $\alpha + \beta x = 0$ and hence $\pi(x) = 1/(1+1) = 0.5$

Logistic regression in one dimension

Data that has a sharp survival cut off point between patients who live or die should have a large value of β .



Data with a lengthy transition from survival to death should have a low value of β .



Types of Logistic Regression

- Binary Logistic Regression
 - The categorical response has only two possible outcomes. Example: Spam or Not
- Multinomial Logistic Regression
 - Three or more categories without ordering. Example: Predicting which food is preferred more (Veg, Non-Veg, Vegan)
- Ordinal Logistic Regression
 - Three or more categories with ordering. Example: Movie rating from 1 to 5

Decision Boundary

- To predict which class a data belongs, a threshold can be set. Based upon this threshold, the obtained estimated probability is classified into classes.
- Say, if $\text{predicted_value} \geq 0.5$, then classify email as spam else as not spam.
- Decision boundary can be **linear** or **non-linear**.
- Polynomial order can be increased to get complex decision boundary.

Decision Tree

Decision-Tree

- a decision tree consists of
 - **Nodes:**
 - test for the value of a certain attribute
 - **Edges:**
 - correspond to the outcome of a test
 - connect to the next node or leaf
 - **Leaves:**
 - terminal nodes that predict the outcome

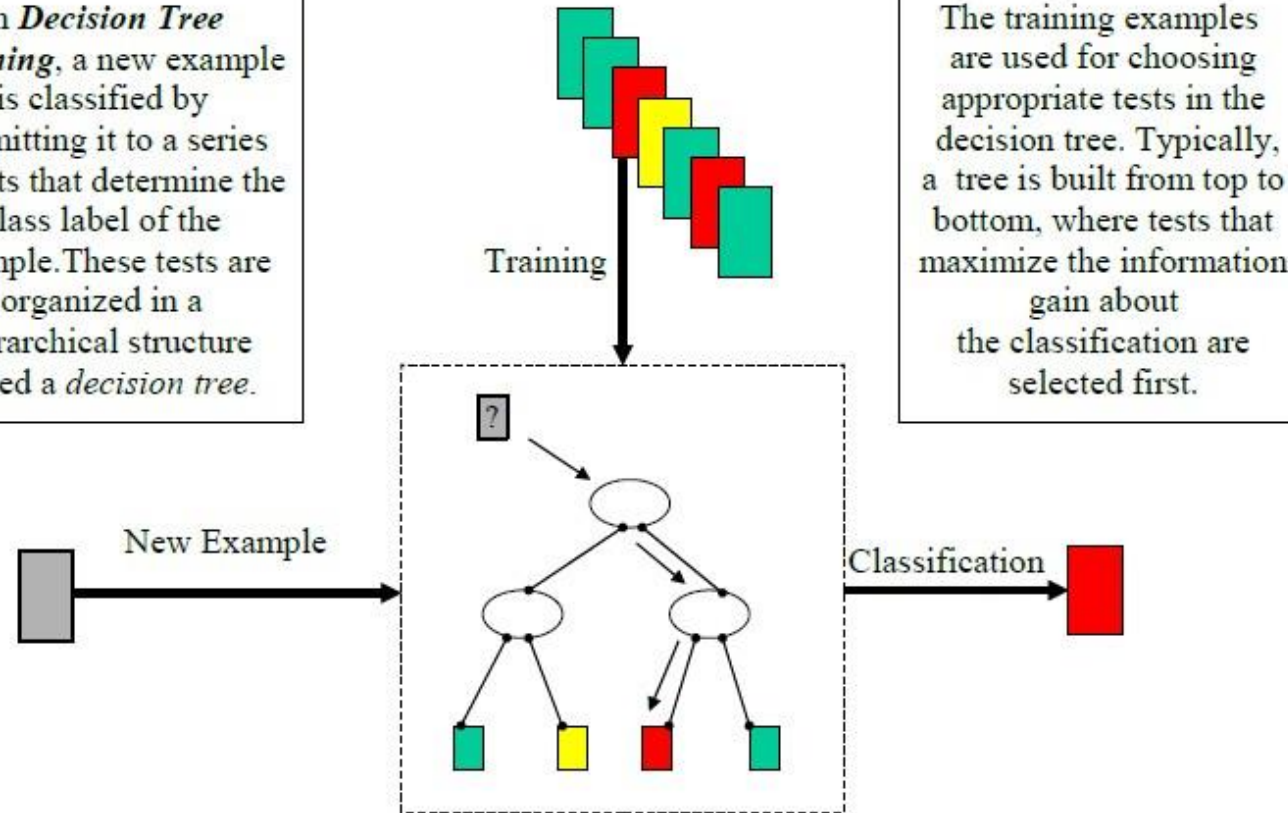
to classify an example:

1. start at the root
2. perform the test
3. follow the edge corresponding to outcome
4. goto 2. unless leaf
5. predict that outcome associated with the leaf



Decision-Tree Learning

In *Decision Tree Learning*, a new example is classified by submitting it to a series of tests that determine the class label of the example. These tests are organized in a hierarchical structure called a *decision tree*.



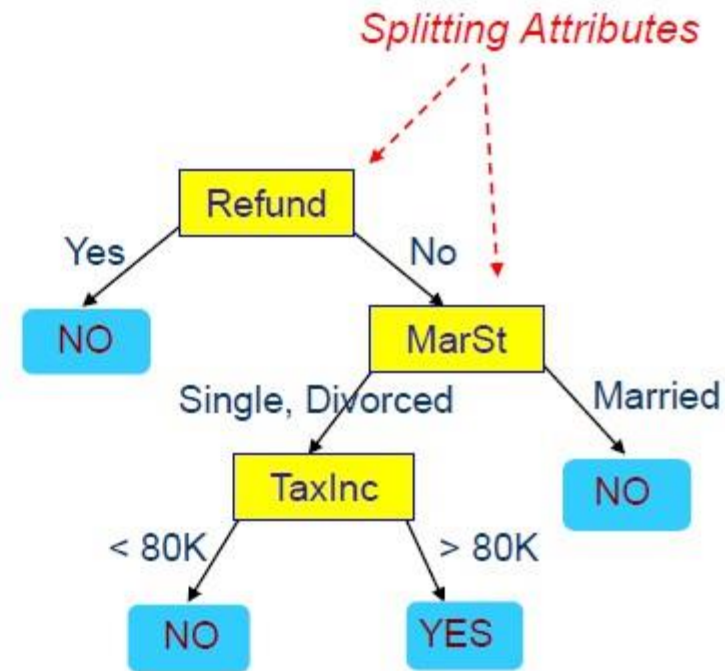
The training examples are used for choosing appropriate tests in the decision tree. Typically, a tree is built from top to bottom, where tests that maximize the information gain about the classification are selected first.

Example of a Decision Tree

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

Training Data

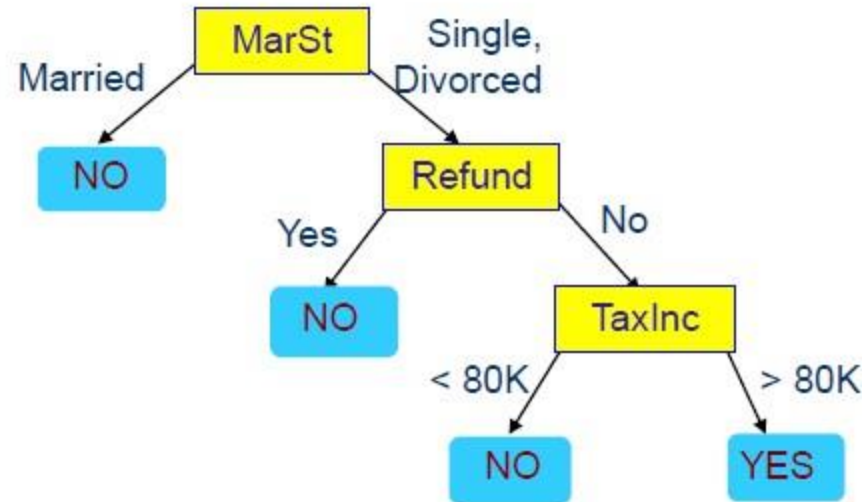


Model: Decision Tree

Another Example of Decision Tree

categorical
categorical
continuous
class

Tid	Refund	Marital Status	Taxable Income	Cheat
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes



There could be more than one tree that fits the same data!

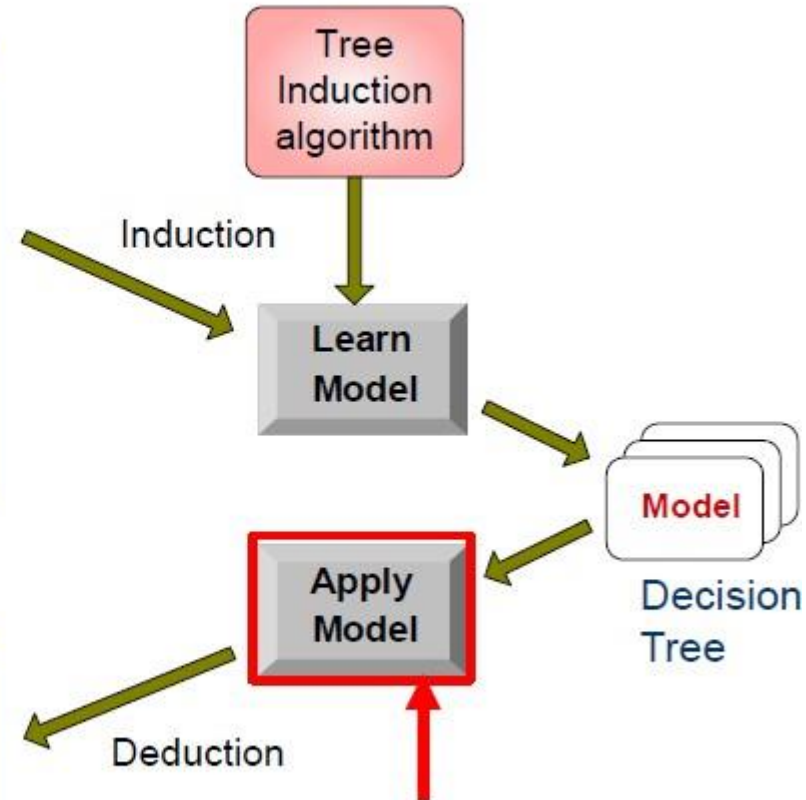
Decision Tree Classification Task

Tid	Attrib1	Attrib2	Attrib3	Class
1	Yes	Large	125K	No
2	No	Medium	100K	No
3	No	Small	70K	No
4	Yes	Medium	120K	No
5	No	Large	95K	Yes
6	No	Medium	60K	No
7	Yes	Large	220K	No
8	No	Small	85K	Yes
9	No	Medium	75K	No
10	No	Small	90K	Yes

Training Set

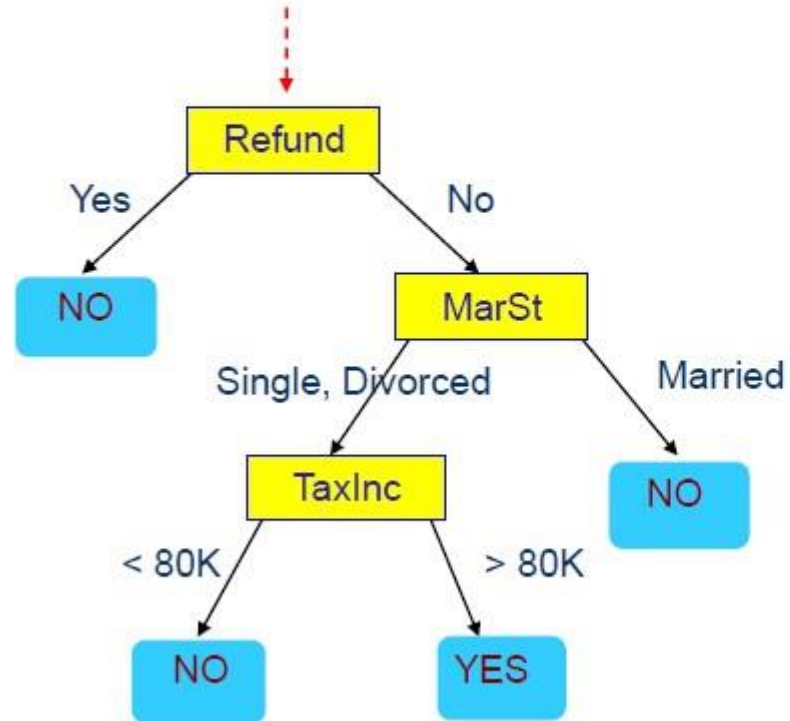
Tid	Attrib1	Attrib2	Attrib3	Class
11	No	Small	55K	?
12	Yes	Medium	80K	?
13	Yes	Large	110K	?
14	No	Small	95K	?
15	No	Large	67K	?

Test Set



Apply Model to Test Data

Start from the root of tree.



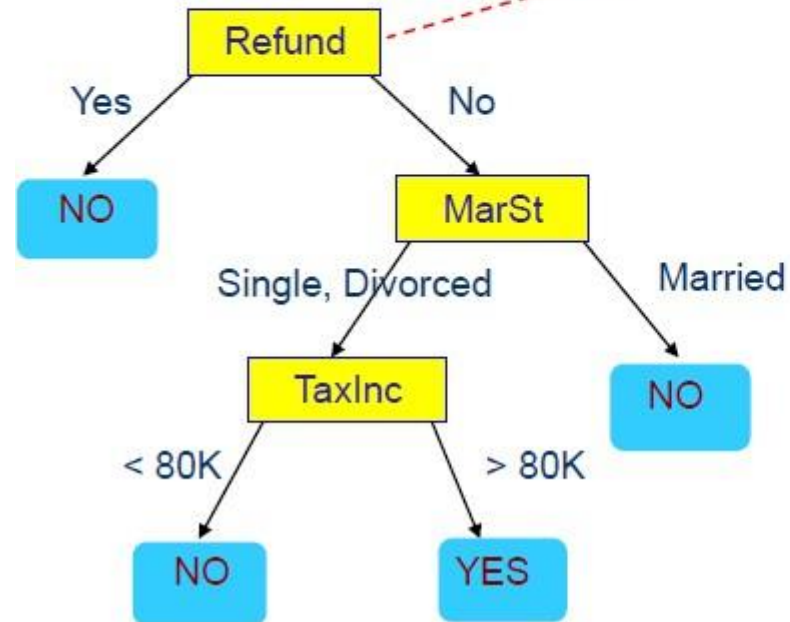
Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?

Apply Model to Test Data

Test Data

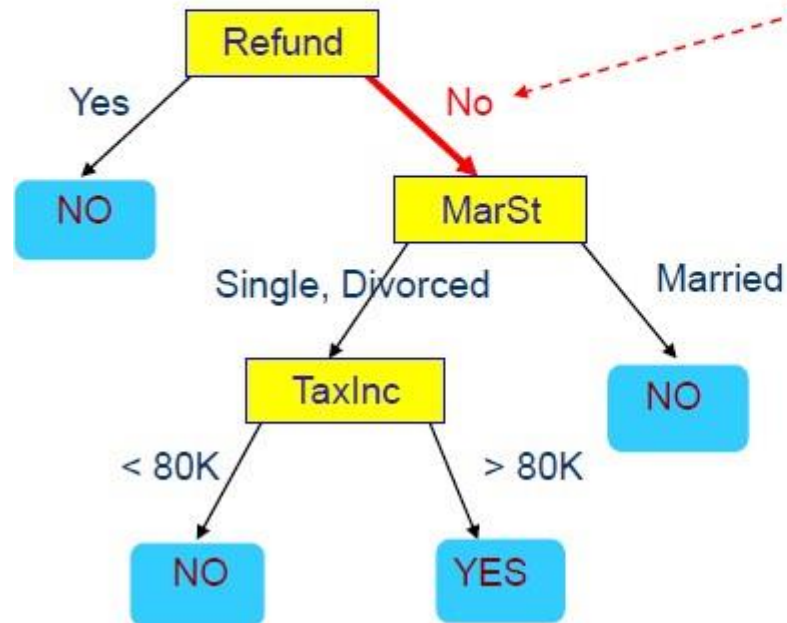
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

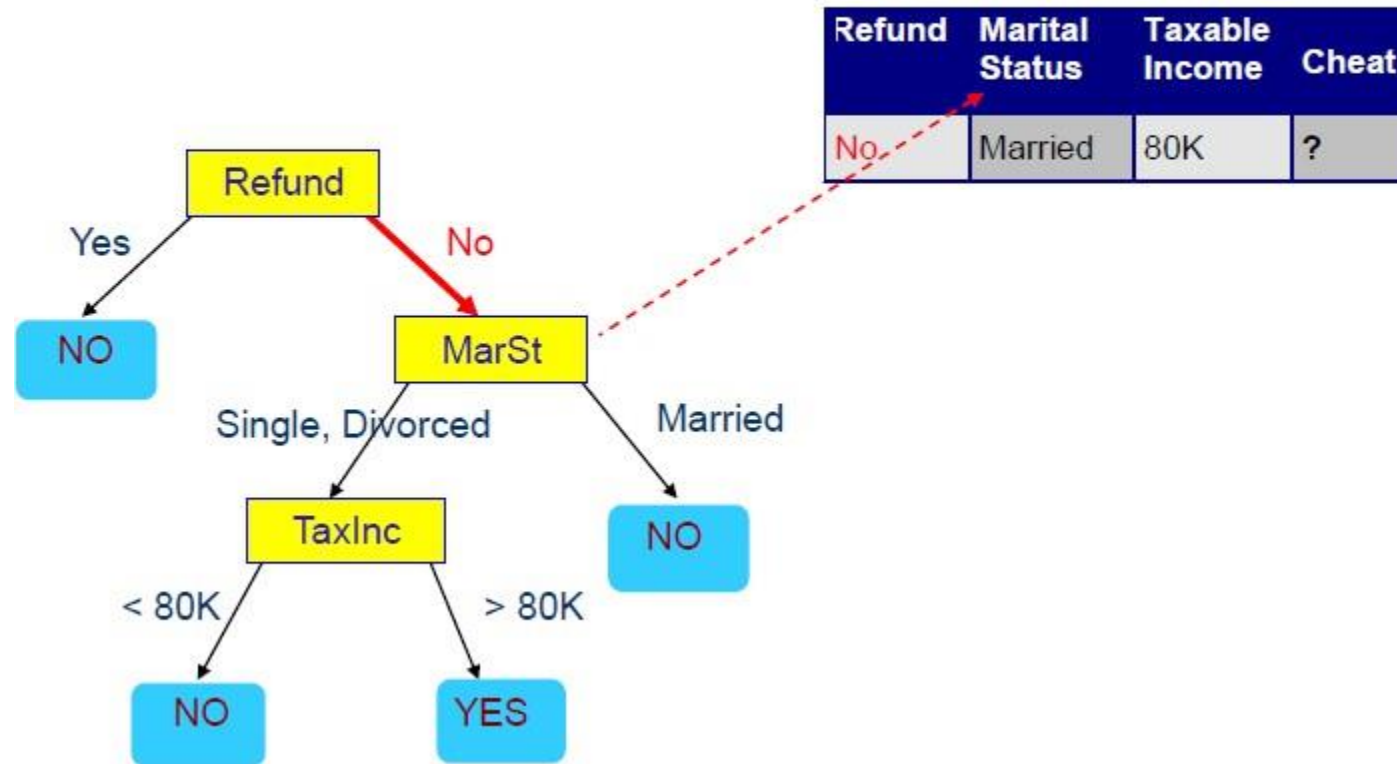
Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

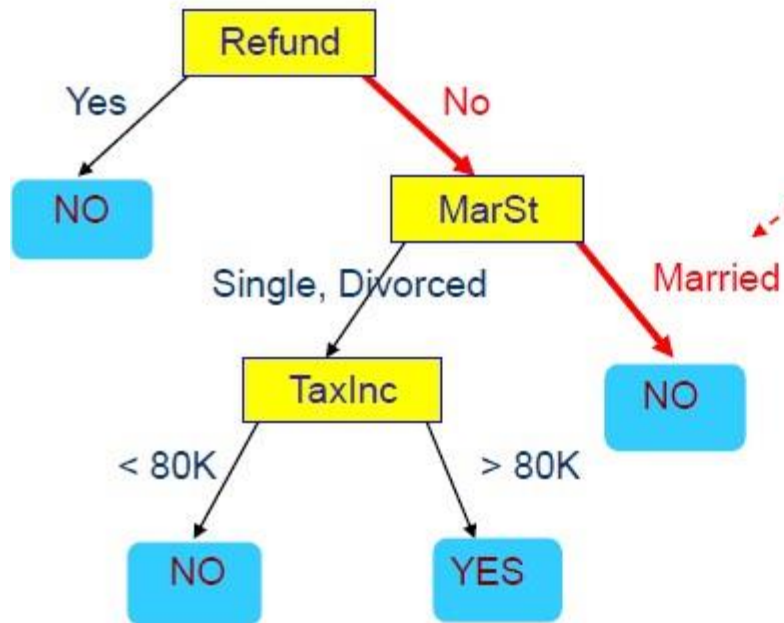
Test Data



Apply Model to Test Data

Test Data

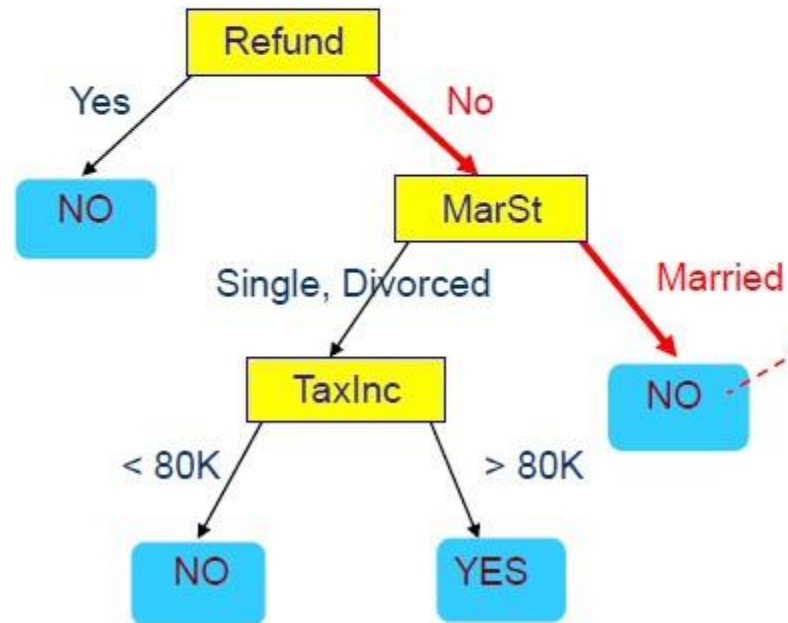
Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Apply Model to Test Data

Test Data

Refund	Marital Status	Taxable Income	Cheat
No	Married	80K	?



Assign Cheat to "No"

Decision Trees Functions:

- Data compression
- Prediction

Decision Tree

- An internal node is a test on an attribute
- A branch represents an outcome of the test, e.g., COLOR = Red
- A leaf node represents a class label or class label distribution
- At each node, one attribute is chosen to split training examples into distinct classes as much as possible
- A new case is classified by following a matching path to a leaf node
- The no. of leaf nodes represent the no of branches.
- Each branch corresponds to a *classification rule*.

Decision Tree Based Classification

- Advantages:
 - Inexpensive to construct
 - Extremely fast at classifying unknown records
 - Easy to interpret for small-sized trees
 - Decision trees provide a clear indication of which features are most important for classification.
 - Decision trees perform classification without requiring much computation.
 - Accuracy is comparable to other classification techniques for many simple data sets

The TDIDT Algorithm

(Top-Down Induction of Decision Trees)

- Many Algorithms:
 - Hunt's Algorithm (one of the earliest)
 - CART
 - ID3 (Iterative Dichotomiser 3), C4.5, C5.0
 - SLIQ, SPRINT

Divide-And-Conquer Algorithms

- Family of decision tree learning algorithms
 - TDIDT: Top-Down Induction of Decision Trees
- Learn trees in a Top-Down fashion:
 - divide the problem in subproblems
 - solve each problem

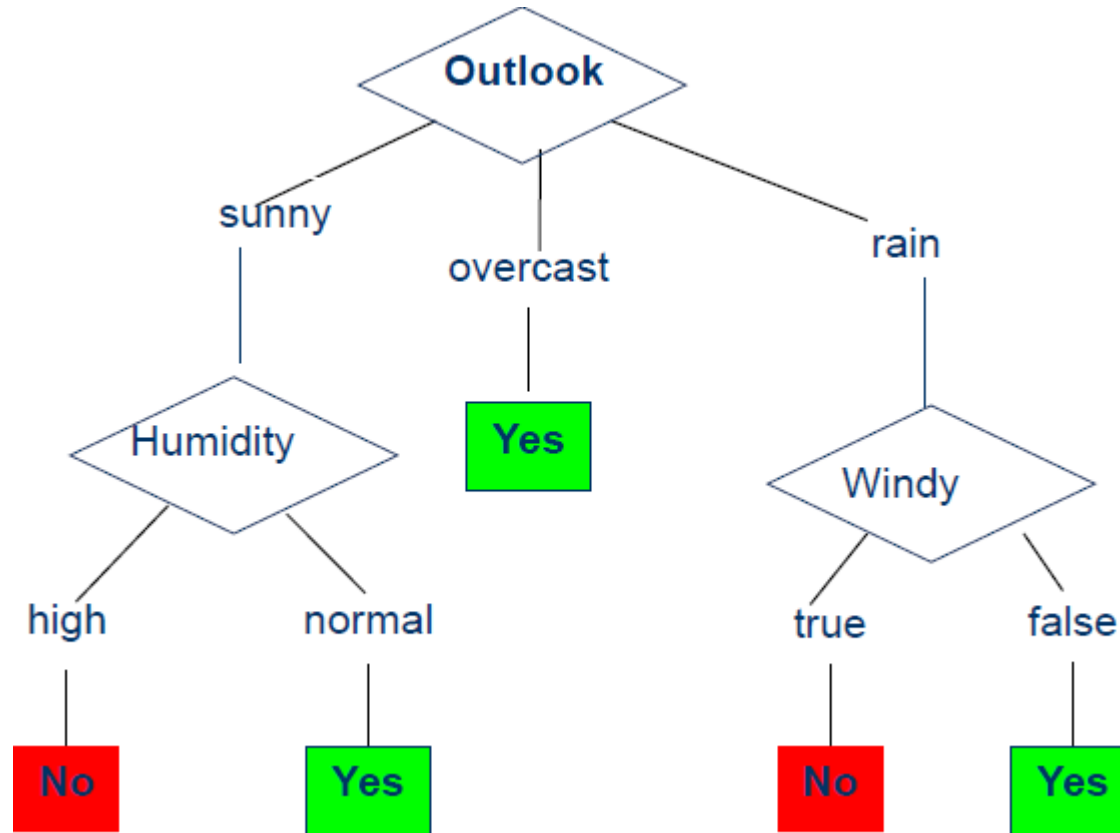
Basic Divide-And-Conquer Algorithm:

1. select a test for root node
Create branch for each possible outcome of the test
2. split instances into subsets
One for each branch extending from the node
3. repeat recursively for each branch, using only instances that reach the branch
4. stop recursion for a branch if all its instances have the same class

Weather Data: Play or not Play? Outlook

Outlook	Temperature	Humidity	Windy	Play?
sunny	hot	high	false	No
sunny	hot	high	true	No
overcast	hot	high	false	Yes
rain	mild	high	false	Yes
rain	cool	normal	false	Yes
rain	cool	normal	true	No
overcast	cool	normal	true	Yes
sunny	mild	high	false	No
sunny	cool	normal	false	Yes
rain	mild	normal	false	Yes
sunny	mild	normal	true	Yes
overcast	mild	high	true	Yes
overcast	hot	normal	false	Yes
rain	mild	high	true	No

Example Tree for “Play?”



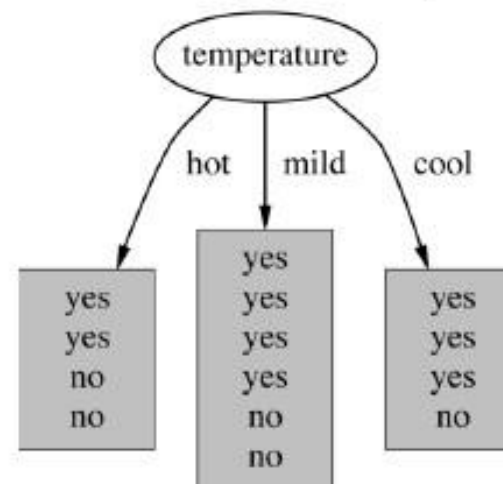
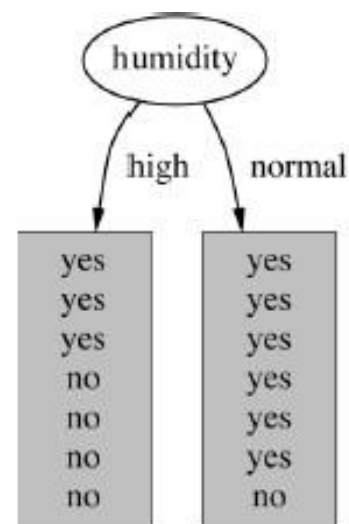
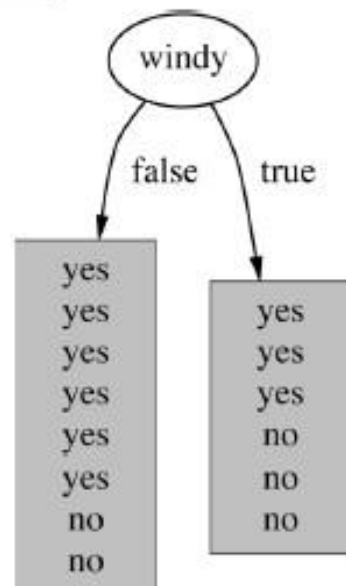
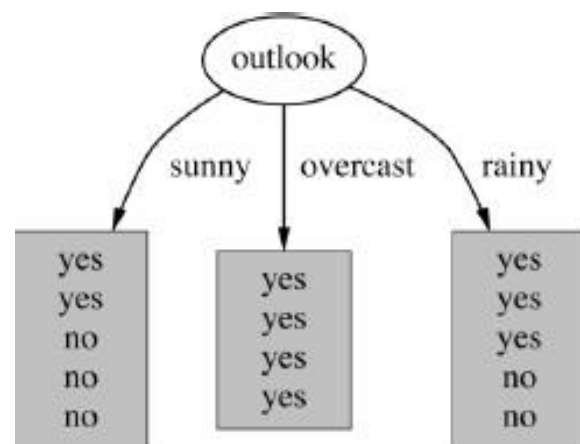
Building Decision Tree

- **Top-down tree construction**
 - At start, all training examples are at the root
 - Partition the examples recursively by choosing one attribute each time
- **Bottom-up tree pruning**
 - Remove subtrees or branches, in a bottom-up manner, to improve the estimated accuracy on new

Choosing the Splitting Attribute

- At each node, available attributes are evaluated on the basis of separating the classes of the training examples. A Goodness function is used for this purpose
- **Typical goodness functions:**
 - information gain (ID3/C4.5)
 - accuracy
 - giniindex
 - others (information gain ratio)

Which attribute to select?



A criterion for attribute selection

- **Which is the best attribute?**
 - The one which will result in the smallest tree
 - Heuristic: choose the attribute that produces the “purest” nodes
- **Popular *impurity criterion: information gain***
 - Information gain increases with the average purity of the subsets that an attribute produces
- **Strategy:** choose attribute that results in greatest information gain

Computing information

- Information is measured in *bits*
 - Given a probability distribution, the info required to predict an event is the distribution's *entropy*
 - Entropy gives the information required in bits (this can involve fractions of bits!)
- Formula for computing the entropy:

$$\text{entropy}(p_1, p_2, \dots, p_n) = -p_1 \log p_1 - p_2 \log p_2 \dots - p_n \log p_n$$

Example: attribute “Outlook”

- “Outlook” = “Sunny”:

$$\text{info}([2,3]) = \text{entropy}(2/5, 3/5) = -2/5 \log(2/5) - 3/5 \log(3/5) = 0.971 \text{ bits}$$

- “Outlook” = “Overcast”:

$$\text{info}([4,0]) = \text{entropy}(1,0) = -1 \log(1) - 0 \log(0) = 0 \text{ bits}$$

Note: $\log(0)$ is not defined, but we evaluate $0 \cdot \log(0)$ as zero



- “Outlook” = “Rainy”:

$$\text{info}([3,2]) = \text{entropy}(3/5, 2/5) = -3/5 \log(3/5) - 2/5 \log(2/5) = 0.971 \text{ bits}$$

- Expected information for attribute:

$$\begin{aligned} \text{info}([2,3], [4,0], [3,2]) &= (5/14) \times 0.971 + (4/14) \times 0 + (5/14) \times 0.971 \\ &= 0.693 \text{ bits} \end{aligned}$$

Computing the information gain

- Information gain:

(information before split) – (information after split)

$$\text{gain("Outlook")} = \text{info}([9,5]) - \text{info}([2,3],[4,0],[3,2]) = 0.940 - 0.693 \\ = 0.247 \text{ bits}$$

- Information gain for attributes from weather data:

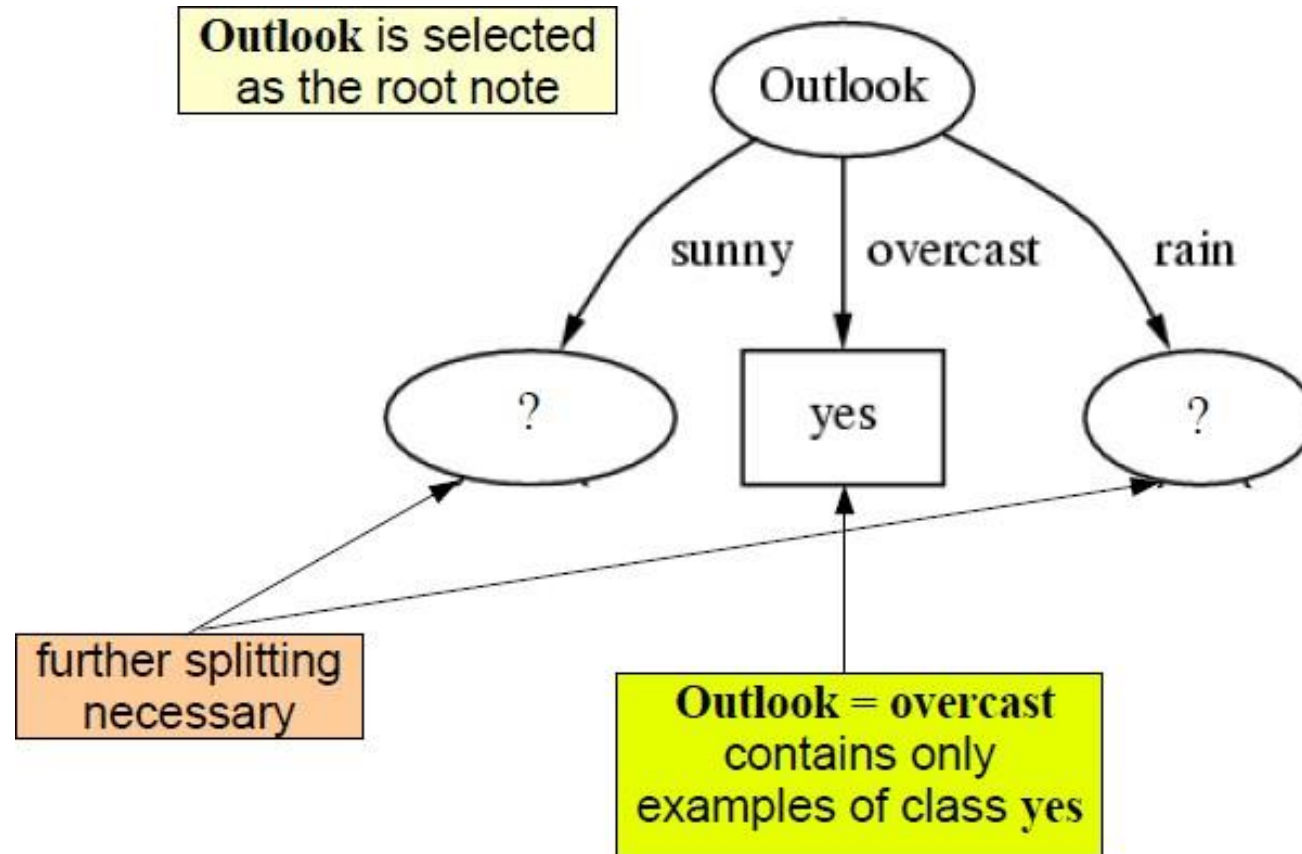
$$\text{gain("Outlook")} = 0.247 \text{ bits}$$

$$\text{gain("Temperature")} = 0.029 \text{ bits}$$

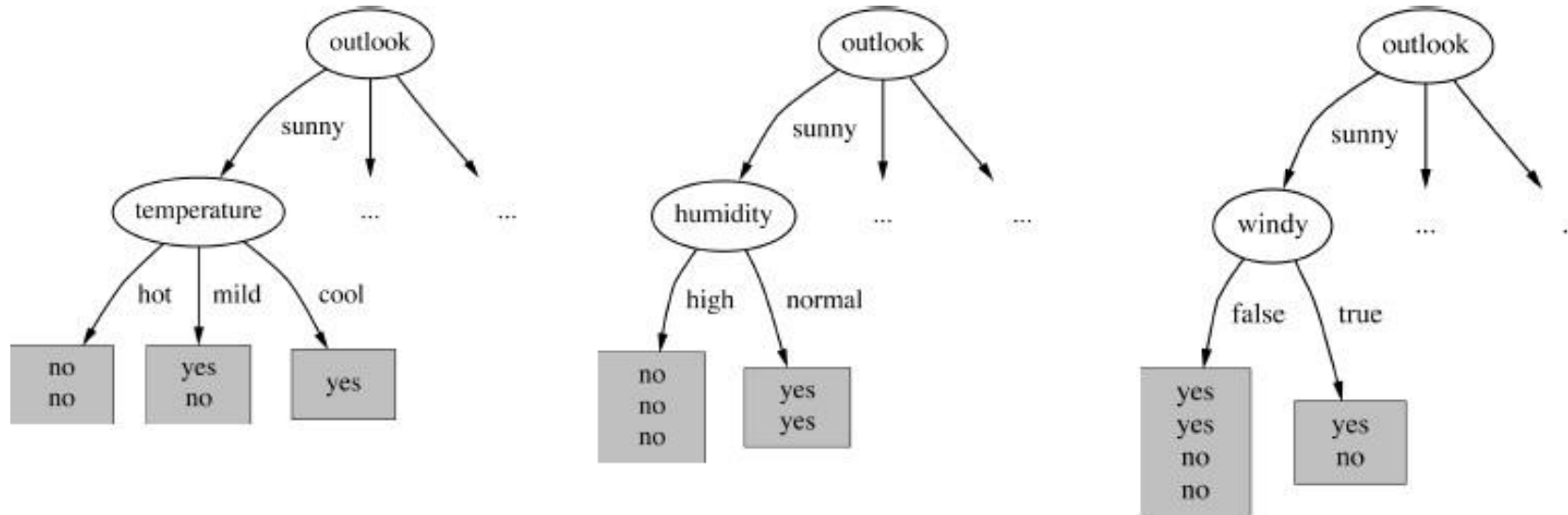
$$\text{gain("Humidity")} = 0.152 \text{ bits}$$

$$\text{gain("Windy")} = 0.048 \text{ bits}$$

Cont'd



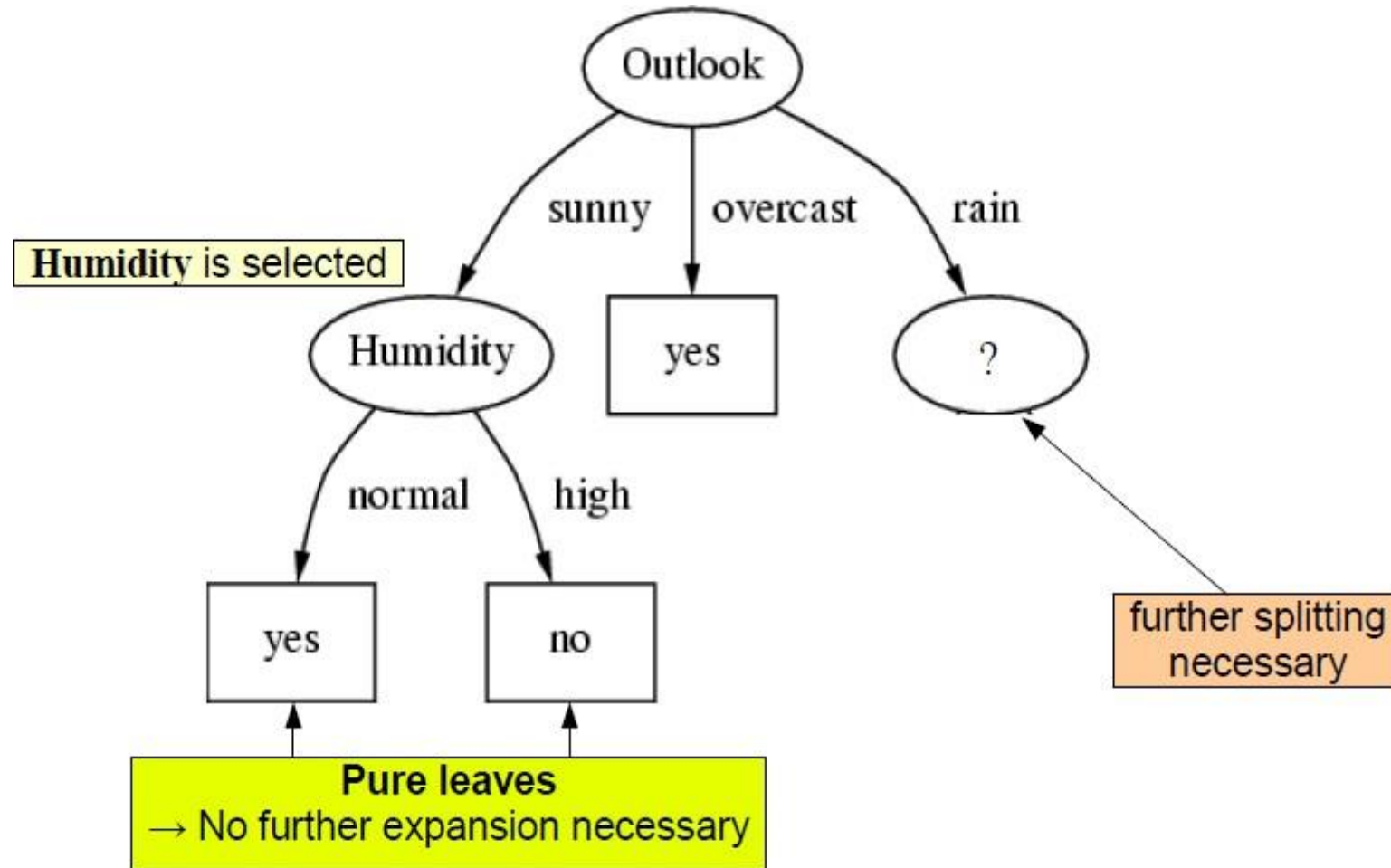
Continuing to split



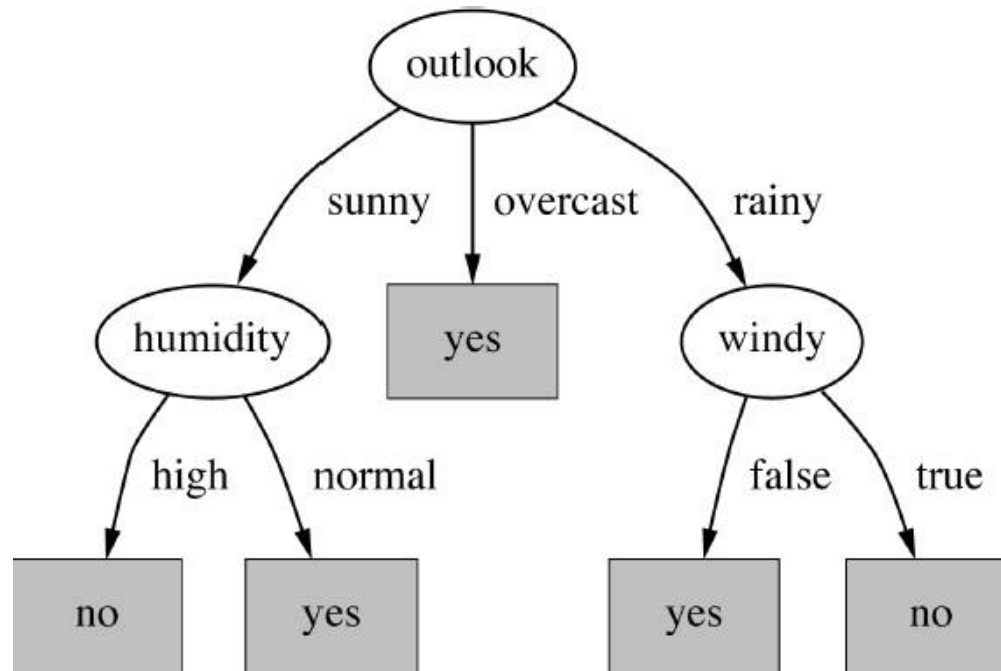
$\text{Gain}(\text{Temperature}) = 0.571 \text{ bits}$
 $\text{Gain}(\text{Humidity}) = 0.971 \text{ bits}$
 $\text{Gain}(\text{Windy}) = 0.020 \text{ bits}$

Humidity is selected

Continuing to split



The final decision tree



Note: not all leaves need to be pure; sometimes identical instances have different classes

- Splitting stops when data can't be split any further

Missing values

- Examples are classified as usual
 - if we are lucky, attributes with missing values are not tested by the tree
- If an attribute with a missing value needs to be tested:
 - split the instance into fractional instances (*pieces*)
 - one piece for each outgoing branch of the node
 - a piece going down a branch receives a weight proportional to the popularity of the branch
 - weights sum to 1
- Info gain or gain ratio work with fractional instances
 - use sums of weights instead of counts
- during classification, split the instance in the same way
 - Merge probability distribution using weights of fractional instances

Overfitting and Pruning

- The smaller the complexity of a concept, the less danger that it overfits the data
 - A polynomial of degree n can always fit $n+1$ points
- Thus, learning algorithms try to keep the learned concepts simple
 - Note a „perfect“ fit on the training data can always be found for a decision tree! (except when data are contradictory)

Pre-Pruning:

- stop growing a branch when information becomes unreliable

Post-Pruning:

- grow a decision tree that correctly classifies all training data
- simplify it later by replacing some nodes with leafs
- Postpruning preferred in practice—prepruning can “stop early”

Prepruning

- Based on statistical significance test
 - Stop growing the tree when there is no *statistically significant* association between any attribute and the class at a particular node
- Most popular test: *chi-squared test*
- ID3 used chi-squared test in addition to information gain
 - Only statistically significant attributes were allowed to be selected by information gain procedure

Early stopping

- Pre-pruning may stop the growth process prematurely: *early stopping*
- Classic example: XOR/Parity-problem

- No individual attribute exhibits any significant association to the class

	a	b	class
1	0	0	0
2	0	1	1
3	1	0	1
4	1	1	0

→ In a dataset that contains XOR attributes a and b, and several irrelevant (e.g., random) attributes, ID3 can not distinguish between relevant and irrelevant attributes

→ Prepruning won't expand the root node

- Structure is only visible in fully expanded tree

- But:

- XOR-type problems rare in practice
- prepruning is faster than postpruning

Post-Pruning

- basic idea
 - first grow a full tree to capture all possible attribute interactions
 - later remove those that are due to chance

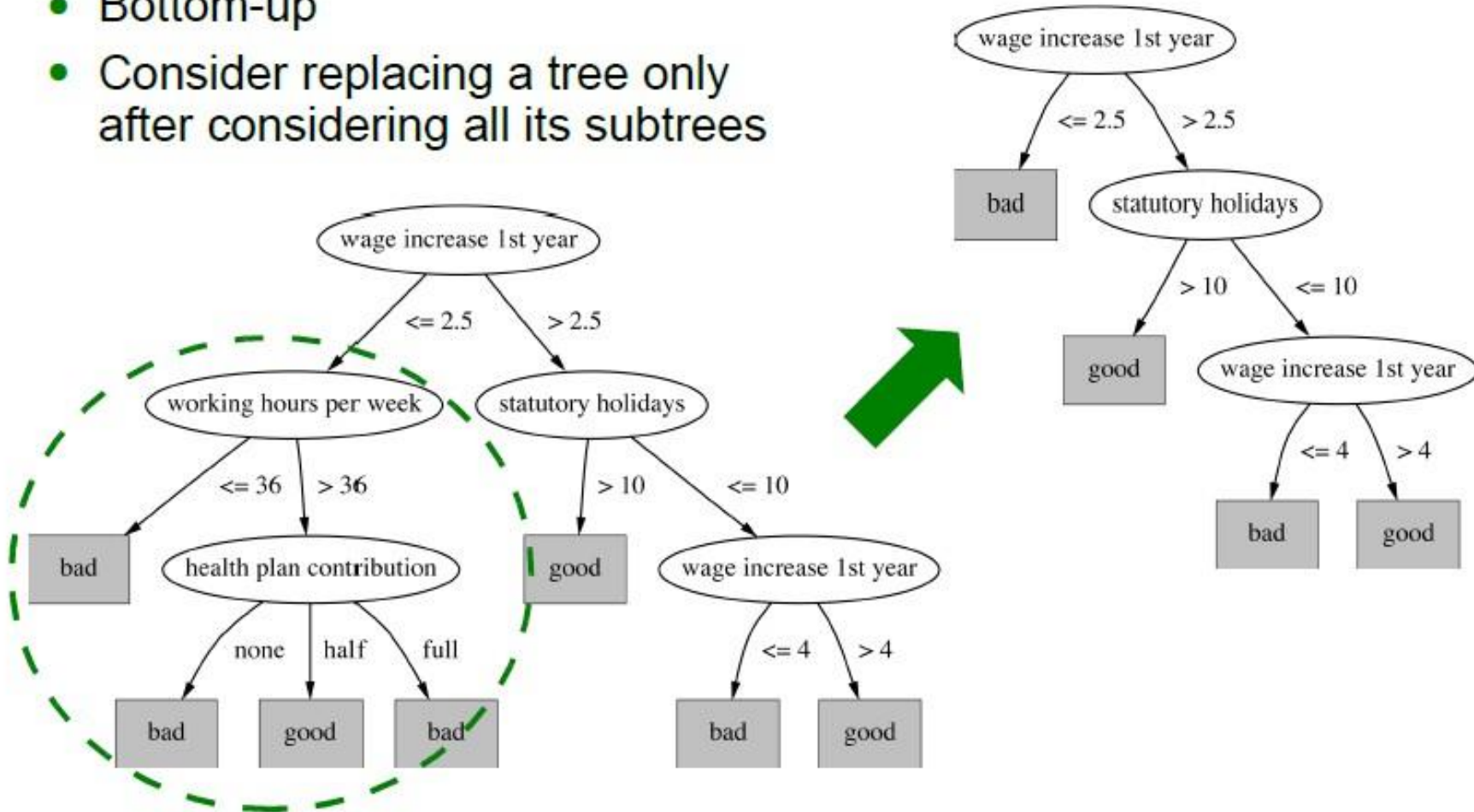
- 1.learn a complete and consistent decision tree that classifies all examples in the training set correctly
- 2.as long as the performance increases
 - try simplification operators on the tree
 - evaluate the resulting trees
 - make the replacement the results in the best estimated performance
- 3.return the resulting decision tree

Postpruning

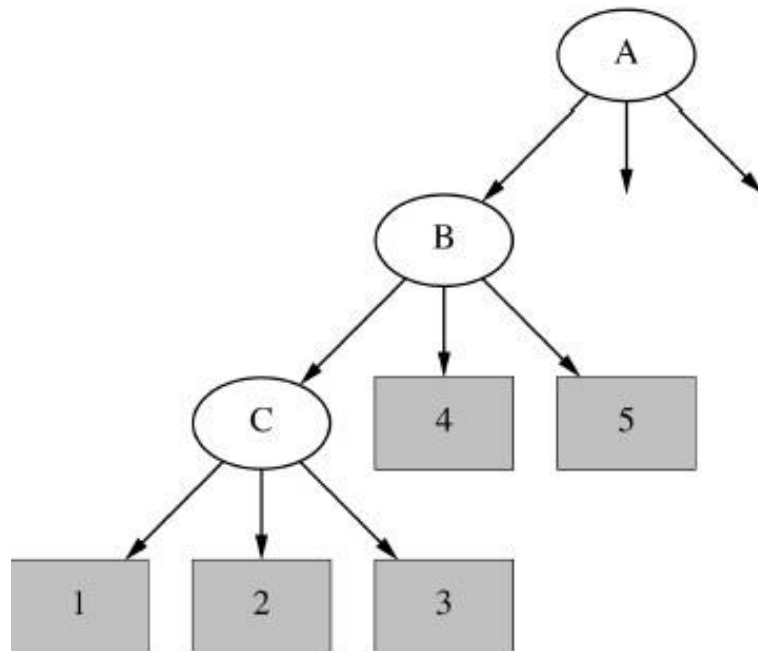
- Two subtree simplification operators
 - Subtree replacement
 - Subtree raising
- Possible performance evaluation strategies
 - error estimation
 - on separate pruning set („reduced error pruning“)
 - with confidence intervals (C4.5's method)
 - significance testing
 - MDL principle

Subtree replacement

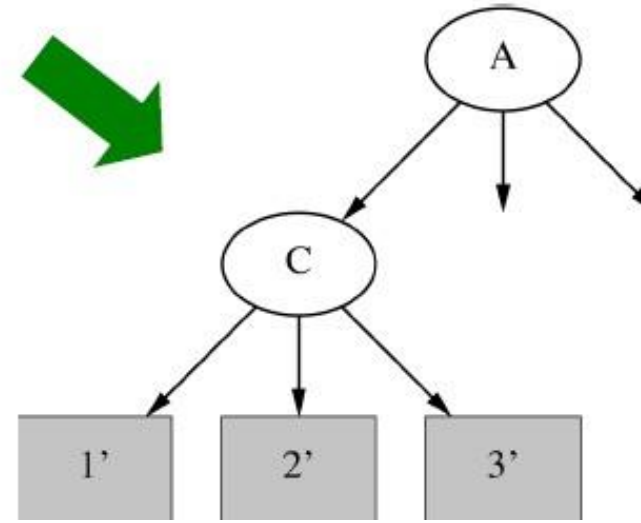
- Bottom-up
- Consider replacing a tree only after considering all its subtrees



Subtree raising



- Delete node B
- Redistribute instances of leaves 4 and 5 into C



Model Trees

- In a Leaf node
 - Classification Trees predict a class value
 - Regression Trees predict the average value of all instances in the model
 - Model Trees use a linear model for making the predictions
 - growing of the tree is as with Regression Trees
- Linear Model:
 - $LM(x) = \sum w_i v_i(x)$ where $v_i(x)$ is the value of attribute A_i for example x and w_i is a weight
 - The attributes that have been used in the path of the tree can be ignored
- Weights can be fitted with standard math packages
 - Minimize the Mean Squared Error $MSE = \sum_j (y_j - r_j)^2$

Stopping Criteria for Tree Induction

- Stop expanding a node when all the records belong to the same class
- Stop expanding a node when all the records have similar attribute values
- Early termination

Summary

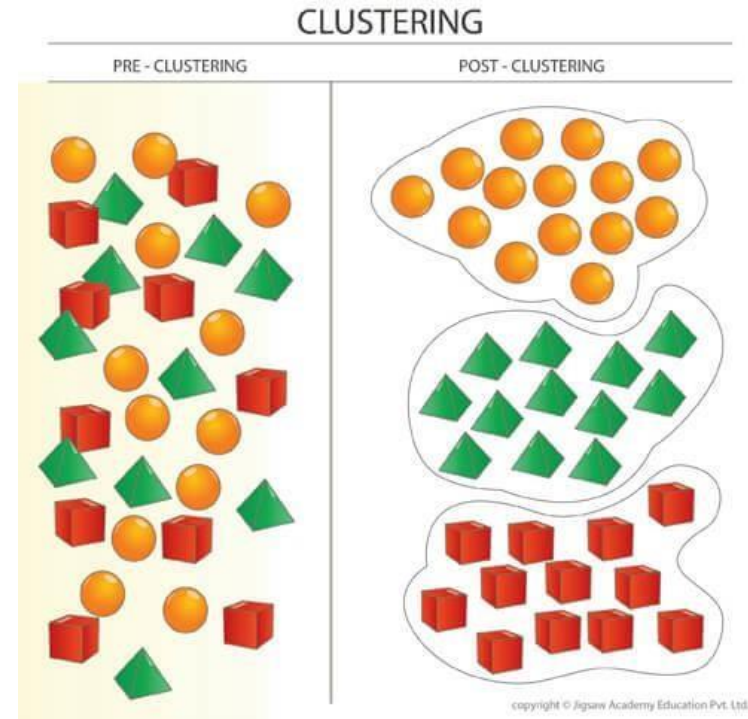
- **Classification Problems** require the prediction of a discrete target value
 - can be solved using decision tree learning
 - iteratively select the best attribute and split up the values according to this attribute
- **Regression Problems** require the prediction of a numerical target value
 - can be solved with regression trees and model trees
 - difference is in the models that are used at the leafs
 - are grown like decision trees, but with different splitting criteria
- **Overfitting** is a serious problem!
 - simpler, seemingly less accurate trees are often preferable
 - evaluation has to be done on separate test sets

K-Means

Clustering

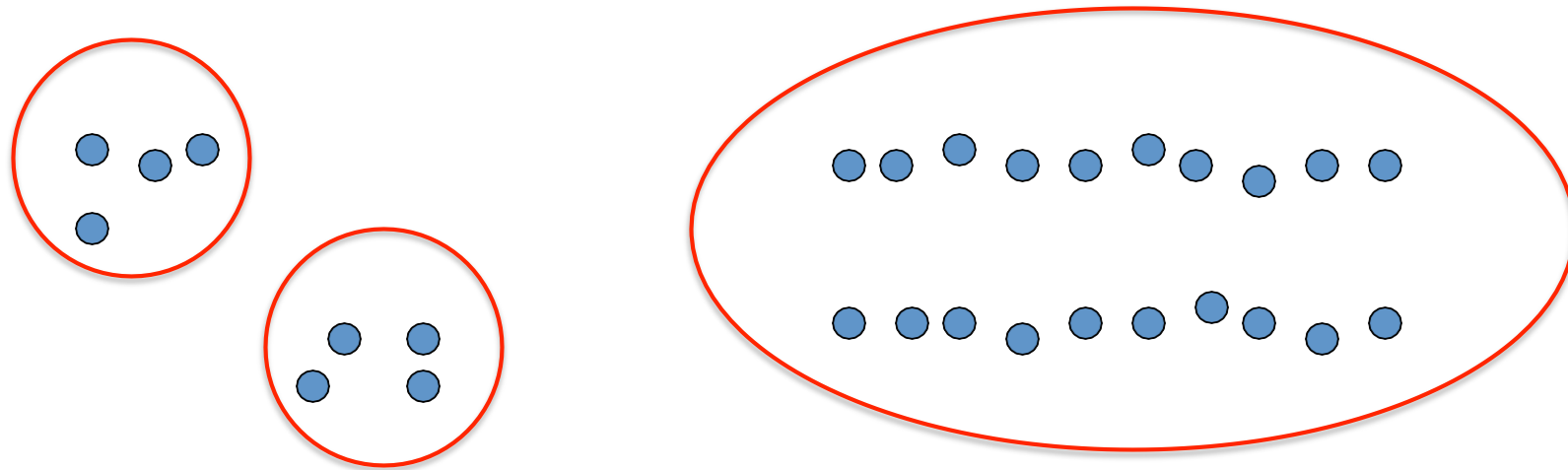
Clustering:

- **Unsupervised learning**
- Requires data, but no labels
- **Detect patterns** e.g. in
 - Group emails or search results
 - Customer shopping patterns
 - Regions of images
- Useful when don't know what you're looking for
- But: can get gibberish



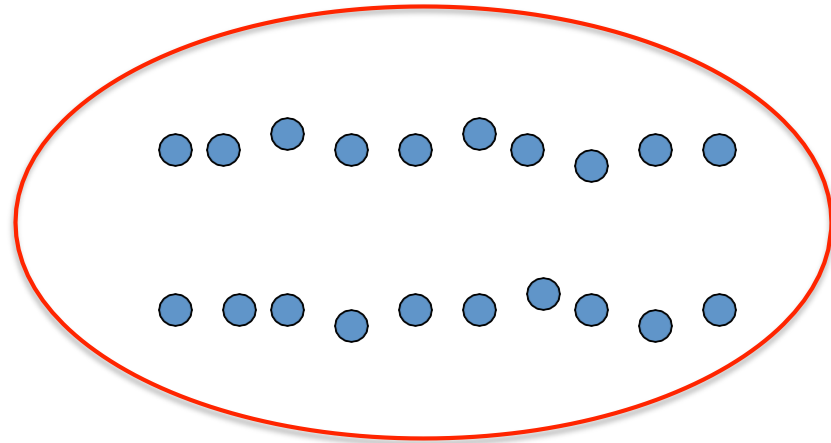
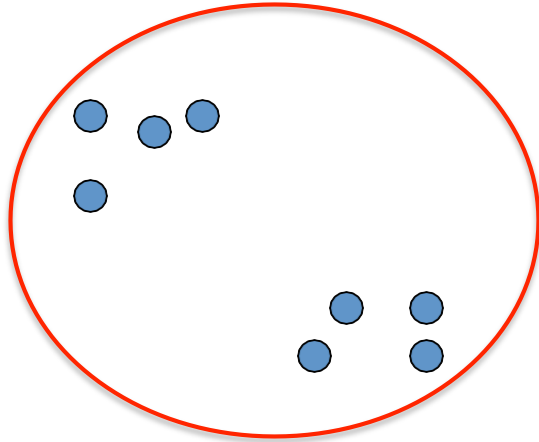
Clustering

- **Basic idea:** group together similar instances
- **Example:** 2D point patterns



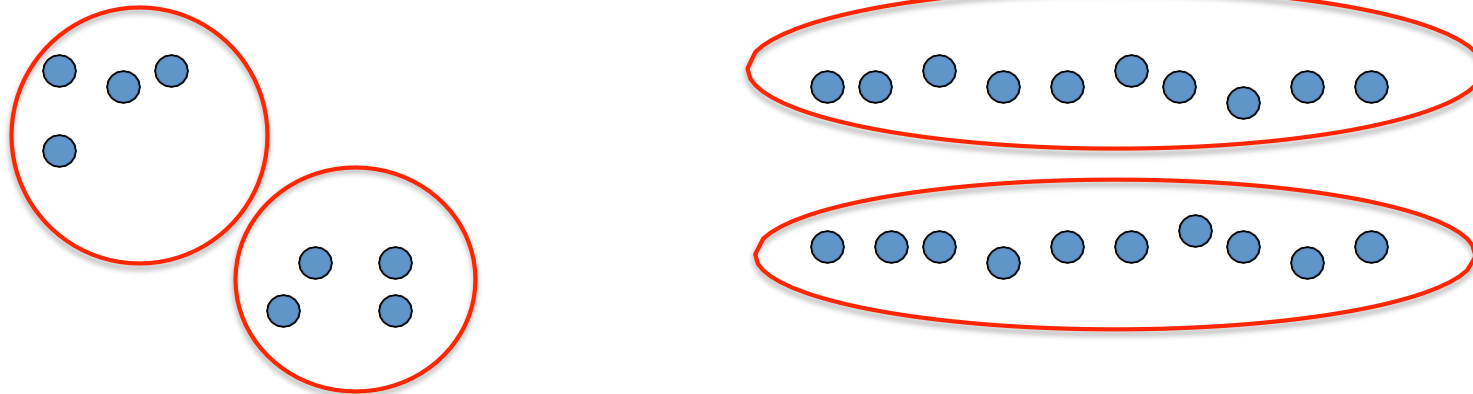
Clustering

- **Basic idea:** group together similar instances
- **Example:** 2D point patterns



Clustering

- Basic idea: group together similar instances
- Example: 2D point patterns

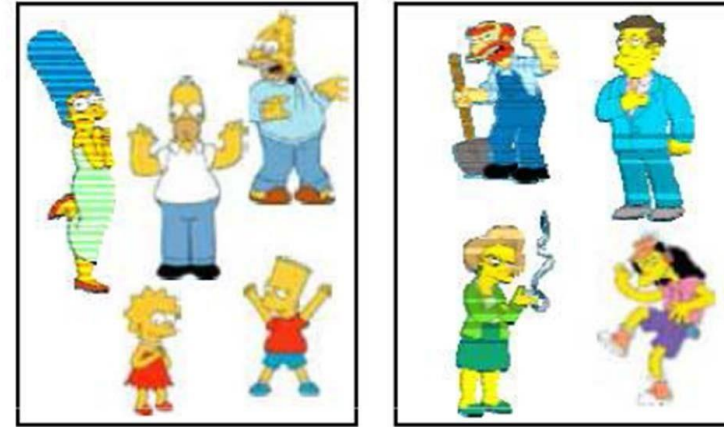


- What could “similar” mean?
 - One option: small Euclidean distance (squared)
$$\text{dist}(x, y) = \|x - y\|_2^2$$
 - Clustering results are crucially dependent on the measure of similarity (or distance) between “points” to be clustered

Clustering algorithms

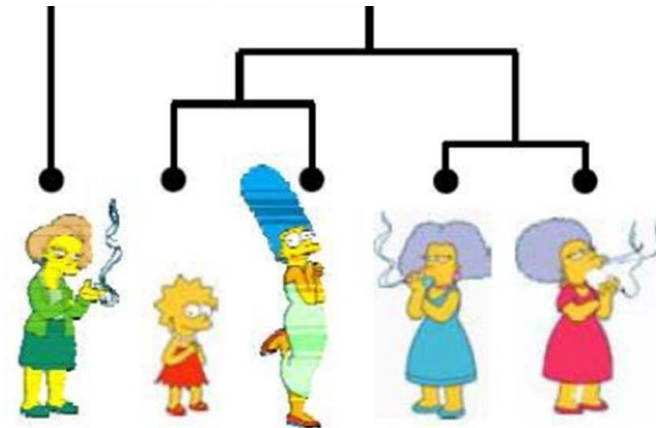
- Partition algorithms (Flat)

- K-means
- Mixture of Gaussian
- Spectral Clustering



- Hierarchical algorithms

- Bottom up – agglomerative
- Top down – divisive



Clustering examples

Image segmentation

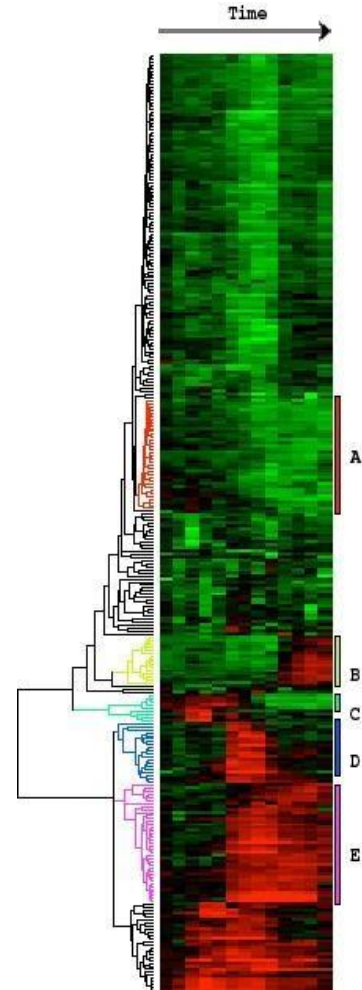
Goal: Break up the image into meaningful or perceptually similar regions



[Slide from James Hayes]

Clustering examples

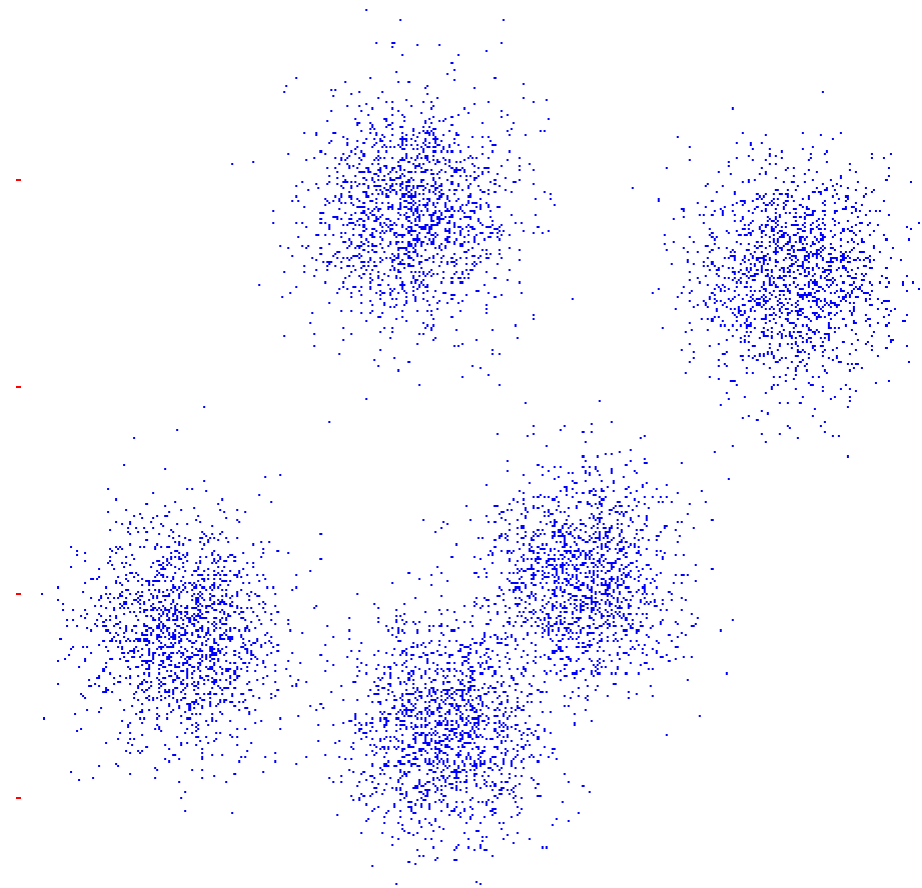
Clustering gene expression data



Eisen et al, PNAS 1998

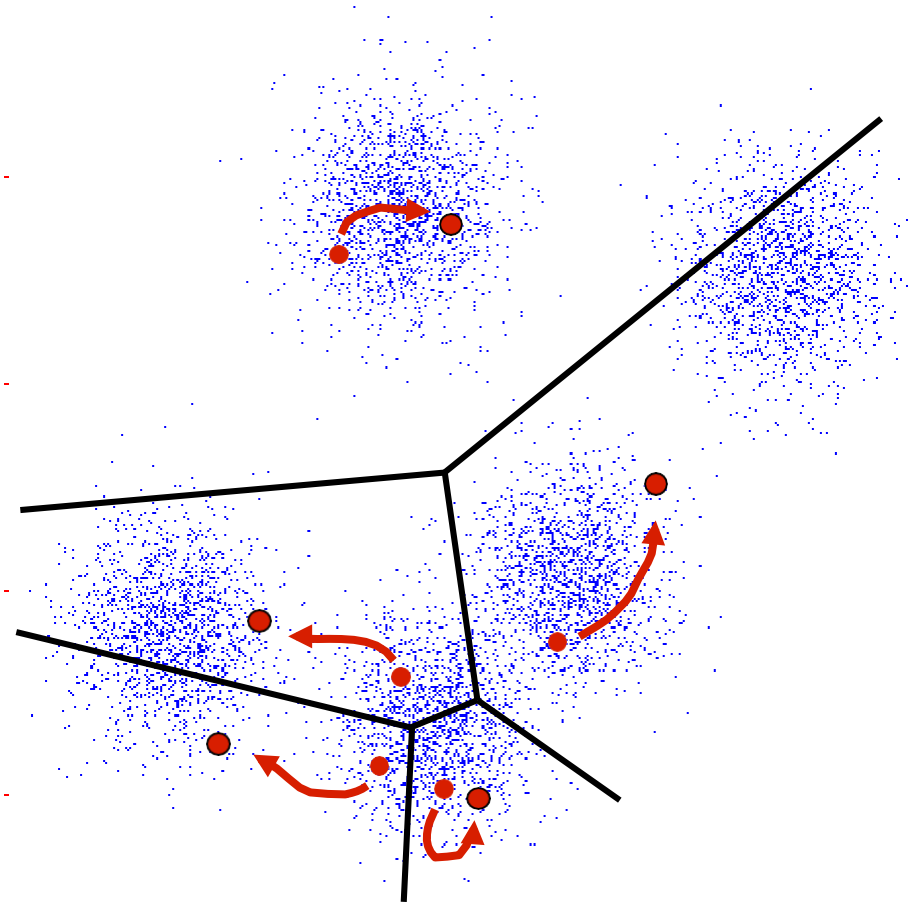
K-Means

- An iterative clustering algorithm
 - **Initialize:** Pick K random points as cluster centers
 - **Alternate:**
 1. Assign data points to closest cluster center
 2. Change the cluster center to the average of its assigned points
 - **Stop** when no points' assignments change

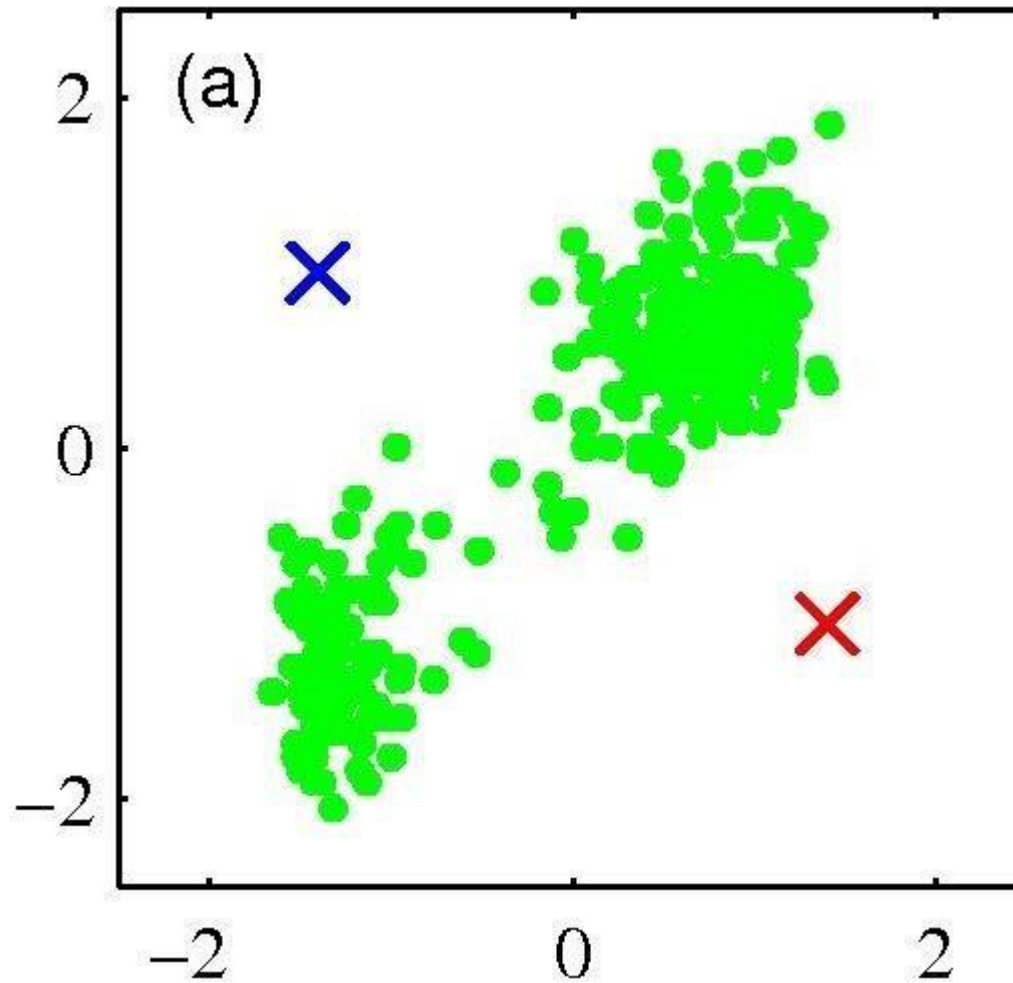


K-Means

- An iterative clustering algorithm
 - **Initialize:** Pick K random points as cluster centers
 - **Alternate:**
 1. Assign data points to closest cluster center
 2. Change the cluster center to the average of its assigned points
 - **Stop** when no points' assignments change



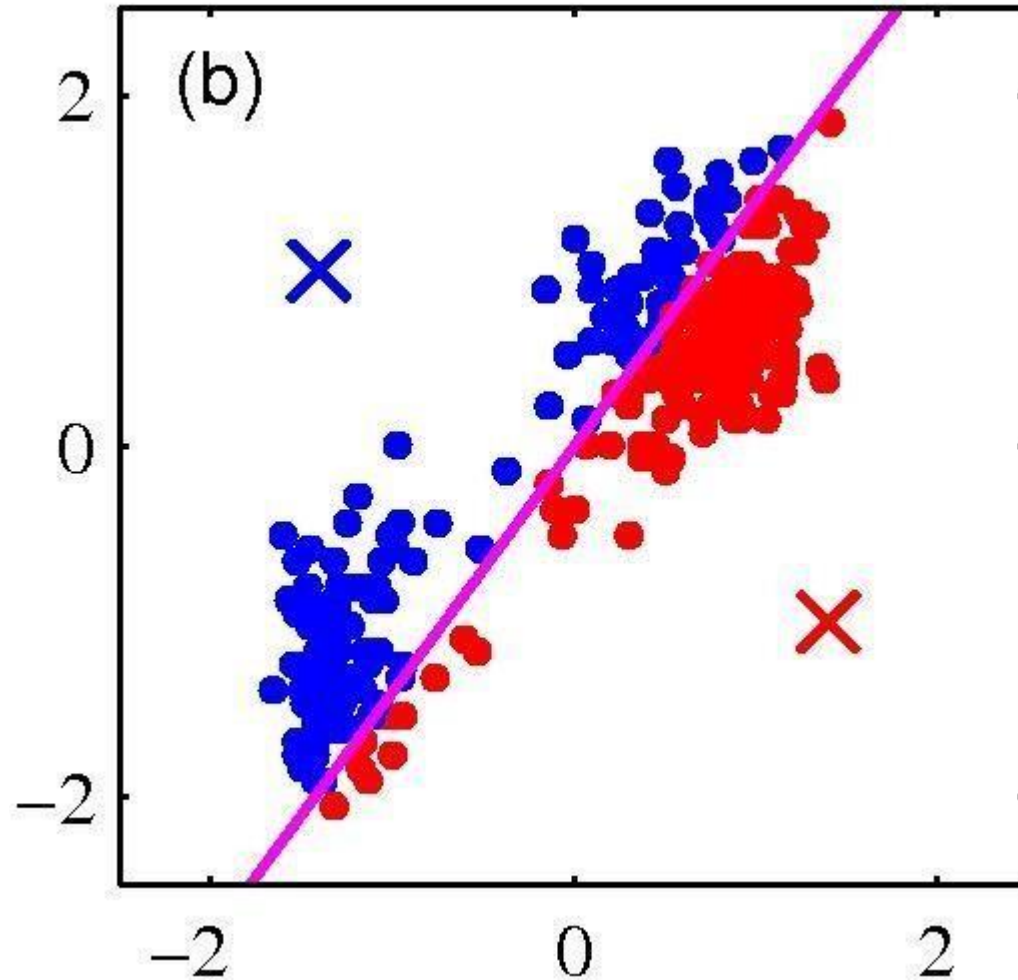
K-means clustering: Example



- Pick K random points as cluster centers (means)

Shown here for $K=2$

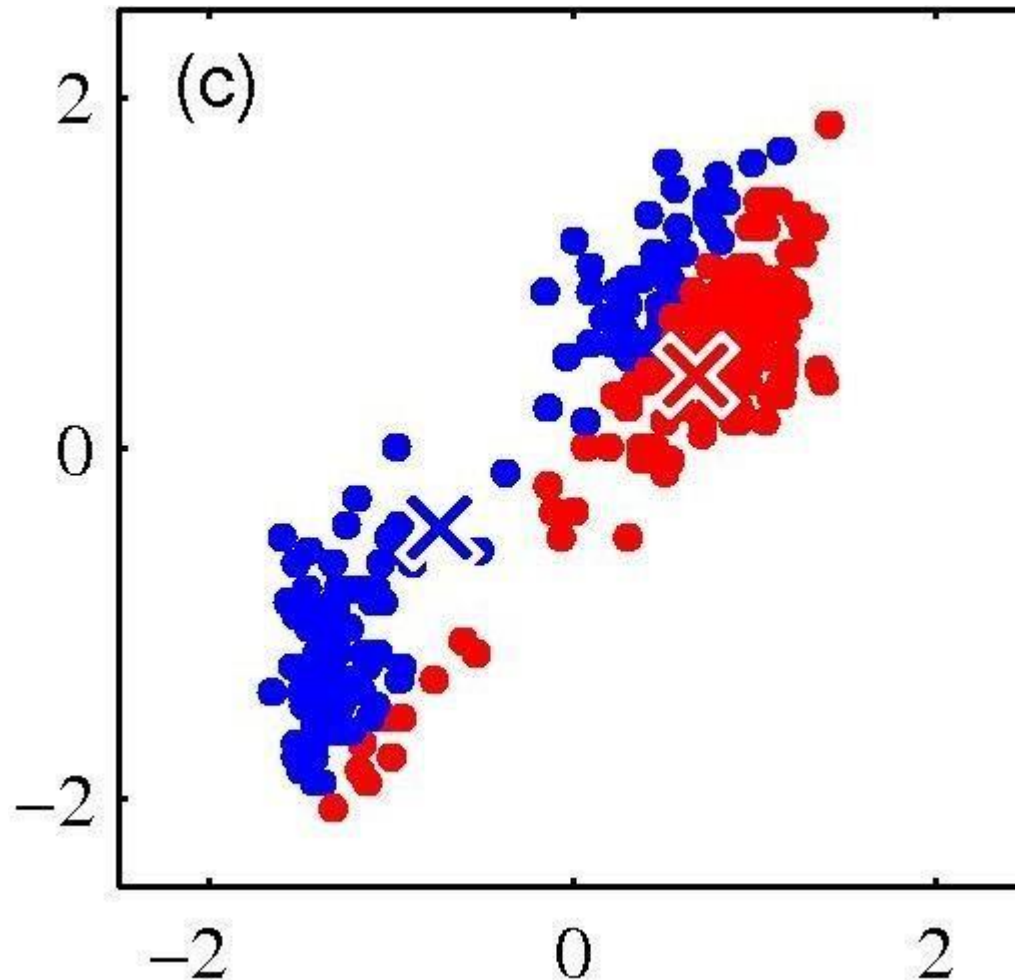
K-means clustering: Example



Iterative Step 1

- Assign data points to closest cluster center

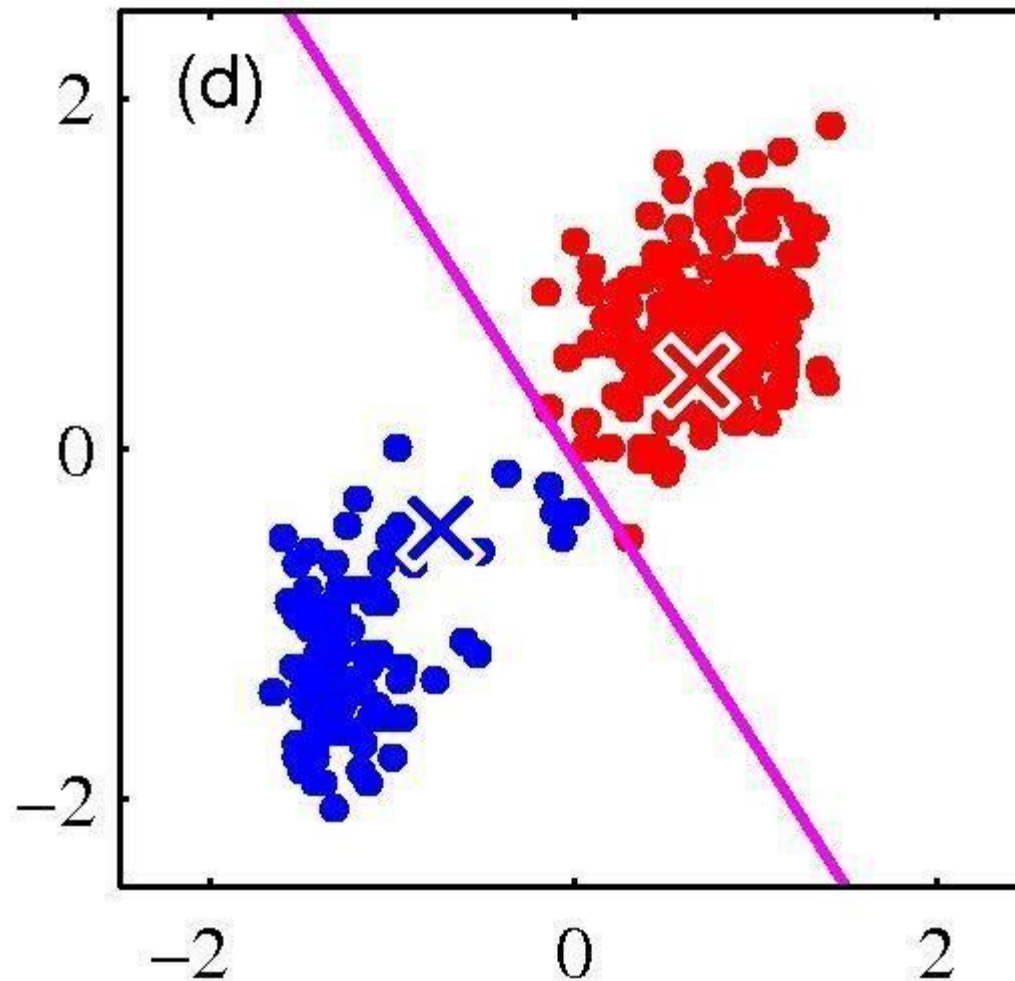
K-means clustering: Example



Iterative Step 2

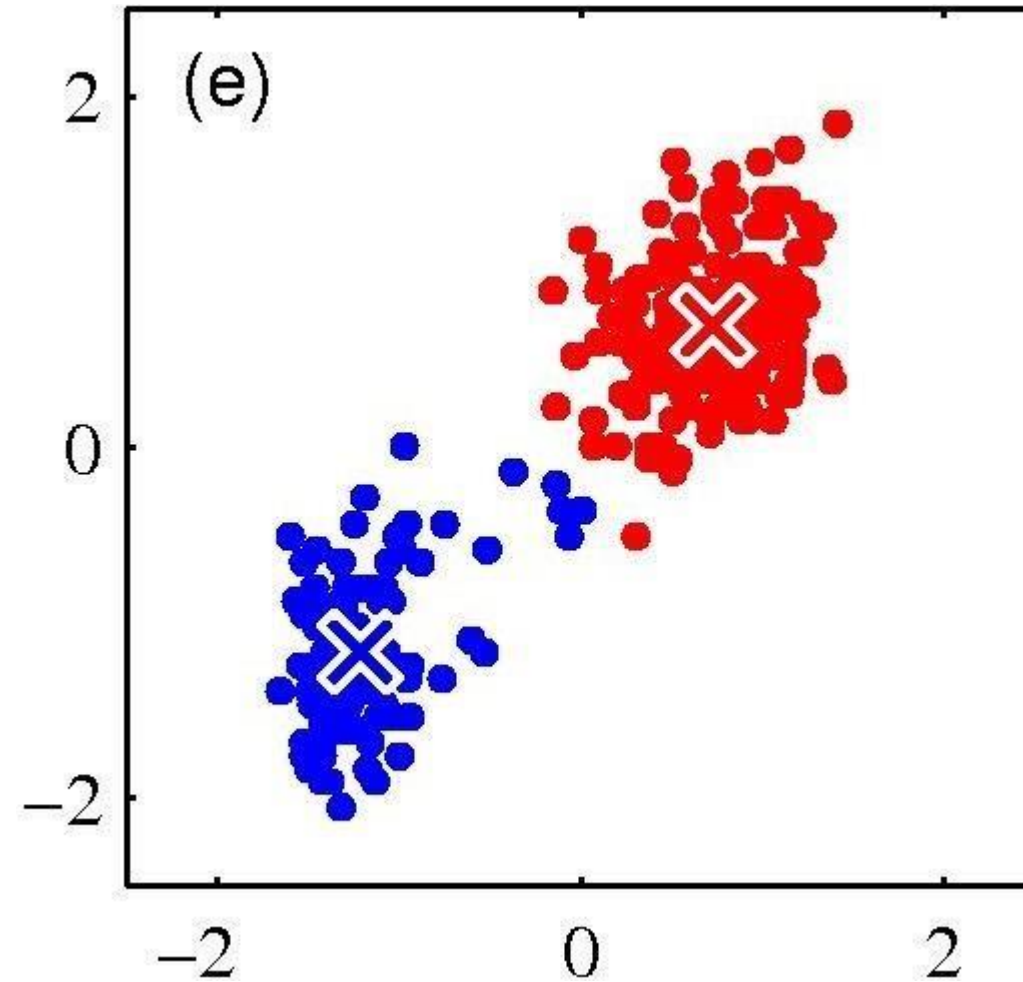
- Change the cluster center to the average of the assigned points

K-means clustering: Example



- Repeat until convergence

K-means clustering: Example



Properties of K-means algorithm

- Guaranteed to converge in a finite number of iterations
- Running time per iteration:
 1. Assign data points to closest cluster center
 $O(KN)$ time
 2. Change the cluster center to the average of its assigned points
 $O(N)$

Example: K-Means for Segmentation

K=2



Goal of Segmentation is to partition an image into regions each of which has reasonably homogenous visual appearance.

Original



Example: K-Means for Segmentation

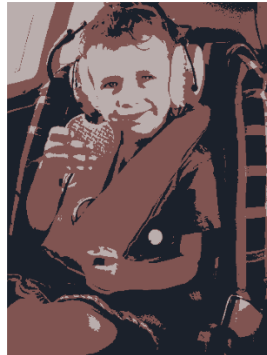
K=2



K=3



Original



Example: K-Means for Segmentation

K=2



K=3



K=10



Original



4%



8%

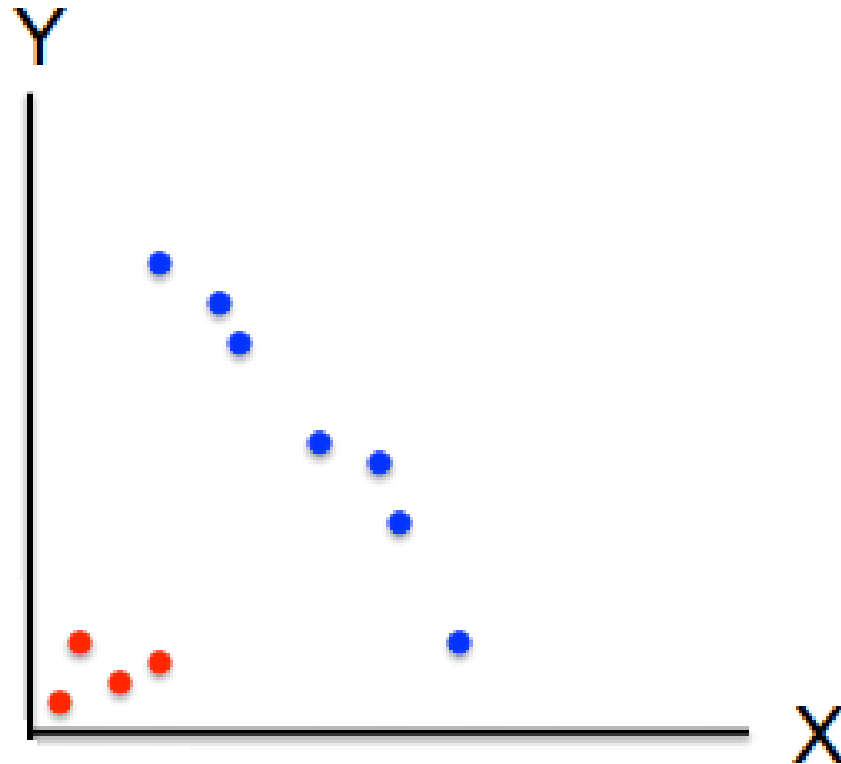


17%



Example 1: K-Means

$K = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$
 $k = 2$



Example 1: K-Means

$$K = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$$
$$k = \underline{2}$$

$$m_1 = 4 \qquad m_2 = 12$$

$$K_1 = \{2, 3, 4\} \qquad K_2 = \{10, 11, 12, 20, 25, 30\}$$

Example 1: K-Means

$$K = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$$
$$k = \underline{2}$$

$$m_1 = 3$$
$$m_2 = \frac{108}{6} = 18$$

$$K_1 = \{2, 3, 4, 10\} \quad K_2 = \{11, 12, 20, 25, 30\}$$

Example 1: K-Means

$$K = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$$
$$k = \underline{2}$$

$$m_1 = 4.75 \qquad m_2 = 19.6$$

$$K_1 = \{2, 3, 4, \underbrace{10, 11, 12}_3\} \quad K_2 = \{20, 25, 30\}$$

Example 1: K-Means

$$K = \{2, 3, 4, 10, 11, 12, 20, 25, 30\}$$
$$k = \underline{2}$$

$$m_1 = 7 \qquad m_2 = 25$$

$$K_1 = \{2, 3, 4, 10, 11, 12\}$$
$$K_2 = \{20, 25, 30\}$$



$$m_1 = 7 \qquad m_2 = 25$$

Example 2: K-Means

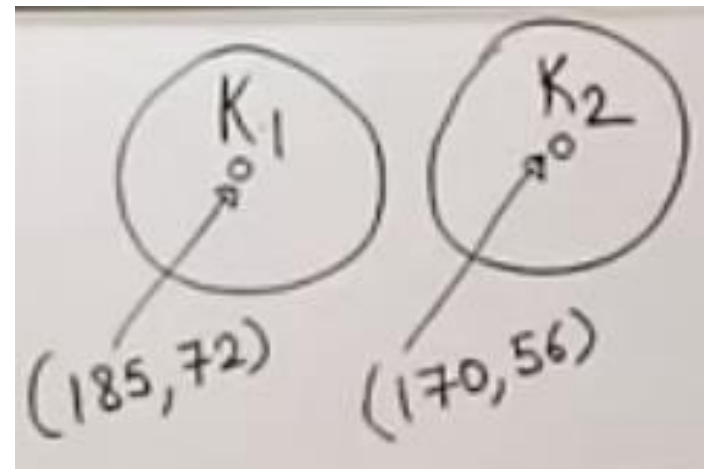
	Height	weight
①	185	72
②	170	56
③	168	60
④	179	68
⑤	182	72
⑥	188	77
⑦	180	71
⑧	180	70
⑨	183	84
⑩	180	88
⑪	180	67
⑫	177	76

Euclidean Distance

$$\sqrt{(X_0 - X_c)^2 + (Y_0 - Y_c)^2}$$

O = Observed

C = Centeroid



Example 2: K-Means

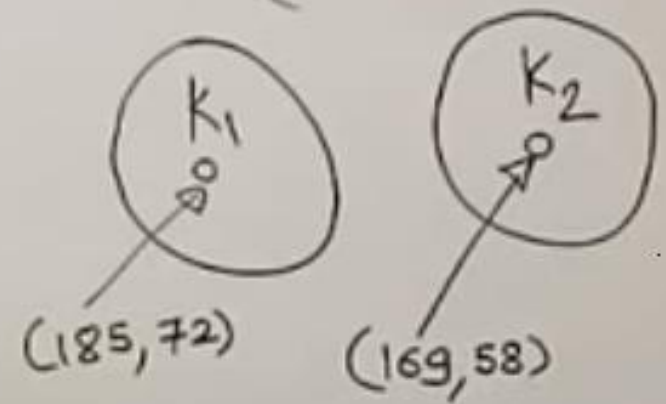
	Height	weight
①	185	72
②	170	56
③	168	60
④	179	68
⑤	182	72
⑥	188	77
⑦	180	71
⑧	180	70
⑨	183	84
⑩	180	88
⑪	180	67
⑫	177	76

$$\begin{aligned} \text{ED for } \textcircled{3} &\rightarrow K_1 \rightarrow \sqrt{(168-185)^2 + (60-72)^2} \\ &\quad \downarrow \\ &\rightarrow K_2 \rightarrow \sqrt{(168-170)^2 + (60-56)^2} \\ &\quad = 4.48 \end{aligned}$$

Example 2: K-Means

	Height	weight
①	185	72
②	170	56
③	168	60
④	179	68
⑤	182	72
⑥	188	77
⑦	180	71
⑧	180	70
⑨	183	84
⑩	180	88
⑪	180	67
⑫	177	76

New Centroid Calculation :-
for $K_2 = \left(\frac{170+168}{2}, \frac{60+56}{2} \right) = (169, 58)$



Example 2: K-Means

	Height	weight
①	185	72
②	170	56
③	168	60
④	179	68
⑤	182	72
⑥	188	77
⑦	180	71
⑧	180	70
⑨	183	84
⑩	180	88
⑪	180	67
⑫	177	76

$$\begin{aligned} \text{E.D for } \rightarrow K_1 &= \sqrt{(179-185)^2 + (68-72)^2} \\ &= (6.32) \\ \text{④} &\rightarrow K_2 = \sqrt{(179-169)^2 + (68-58)^2} \\ &= 14.14 \end{aligned}$$

$$\begin{aligned} K_1 &\rightarrow \{1, 4, 5, 6, 7, 8, 9, 10, 11, 12\} \\ K_2 &\rightarrow \{2, 3\} \end{aligned}$$



Confusion Matrix

- In the field of **machine learning**, a **confusion matrix** is a specific table layout that allows visualization of the performance of an algorithm

	Predicted Negative	Predicted Positive
Actual Negative	True Negative	False Positive
Actual Positive	False Negative	True Positive

Confusion Matrix

- TN is the number of correct predictions that an instance is negative
- FP is the number of incorrect predictions that an instance is positive
- FN is the number of incorrect predictions that an instance is negative
- TP is the number of correct predictions that an instance is positive

Confusion Matrix

- Confusion Matrix from the example of Lecture 2 (without Normalization)

ID	Actual	Predicted
1	1	1
2	0	0
3	1	1
4	1	0

	Negative	Positive
Negative	1	0
Positive	1	2

Confusion Matrix

- Several standard terms have been defined for the 2 class matrix
- The *accuracy* (*AC*) is the proportion of the total number of predictions that were correct

$$Accuracy = \frac{TN + TP}{TN + FN + TP + FP}$$

- Accuracy = 3 / 4 = 75%

Confusion Matrix

- The *recall* or *true positive rate* (*TPR*) is the proportion of positive cases that were correctly identified

$$TPR = \frac{TP}{TP + FN}$$

- The *false positive rate* (*FPR*) is the proportion of negatives cases that were incorrectly classified as positive

$$FPR = \frac{FP}{FP + TN}$$

- $TPR \text{ or recall} = 2 / 3 = 66.7\%$
- $FPR = 0 / 1 = 0 \%$

Confusion Matrix

- The *true negative rate* (*TNR*) is defined as the proportion of negatives cases that were classified correctly,

$$TNR = \frac{TN}{FP + TN}$$

- The *false negative rate* (*FNR*) is the proportion of positives cases that were incorrectly classified as negative

$$FNR = \frac{FN}{FN + TP}$$

- $TNR = 1 / 1 = 100\%$
- $FNR = 1 / 3 = 33.3\%$

Confusion Matrix

$$precision = \frac{tp}{tp + fp}$$

- *precision* (P) is the proportion of the predicted positive cases that were correct,

- precision = $2/2 = 100\%$
- F measure is harmonic mean of precision and recall

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

- $F1 = (2 * 1 * 0.667)/(1+0.667) = 0.8$

Exercise

		Actual	
		Negative	Positive
Predicted	Negative	9760	40
	Positive	140	60